

所属类别	2026 年“华数杯”大学生数学建模竞赛	参赛编号
本科生组		CM2604653

面向算电协同的多区域多目标调度优化研究

摘 要

本文围绕六区域算力基础设施中任务需求、电力参数、新能源与储能状态的耦合关系,构建“需求预测–任务排程–储能控制–局部联合优化”的模型链。任务轨迹按区域、类型和小时汇总为 GPU·h 工作量。以季节朴素预测为基线,使用共享 HistGradientBoosting 模型拟合残差;加权层级校正使 18 条底层序列与系统总量保持一致,合并校准的分割共形方法给出预测区间。独立盲测中,系统加权 WAPE 为 0.8934,18 条底层序列共 432 个逐时点的名义 95% 区间经验覆盖率为 0.9468。实际任务排程采用 CP-SAT,同时满足不可抢占、实时即时开工、单向时延、GPU/IT/设施功率和收尾边界约束;538 条任务完成率为 1.0000。求解状态为 FEASIBLE,故只认定为经独立审计的可行排程。

全时域调度以同输入、同约束的 FIFO 为基线,并用固定权重的有界滚动局部交换启发式完成 50000 条任务的碳感知时空迁移。候选排程完成率为 1.0000,SLA 违约率为 0.0000;相对 FIFO,设施用能成本和购电碳排分别变化 -1.015% 与 -1.929% ,任务加权平均网络时延变化 4.190 ms。候选方案满足完成率和 SLA 要求,但未在全部指标上支配 FIFO,结果体现的是成本、碳排与时延之间的约束权衡。

任务排程确定后,六区域分别求解含充放电互斥、购售电互斥、电能平衡、SOC 动力学和终端 SOC 下界的滚动二进制 MILP。每个区域划分为 14 个 168 h 滚动块和一个 55 h 末块,合计完成 90 次求解。相对无储能基线,净运行成本和购电侧碳排分别变化 $-38,899,970$ CNY 与 -8.851 tCO₂,完整硬约束审计通过 270/270 项。净运行成本按“购电成本减售电收入”定义,负值表示净售电收入;碳排仅计购电侧,模型未显式计入电池退化成本。

Q4 选取 2328–2399 h 的 72 h 窗口,以 Q2 排程为起点,交替执行 12 任务邻域固定–优化搜索和六区域储能追补优化。观测场景净运行成本差为 $-16,061$ CNY;低新能源场景的成本差和购电碳排差分别为 $-41,567$ CNY 与 -7.823 tCO₂,联合压力场景碳排差为 -9.387 tCO₂。全部任务、储能和电网硬约束及确定性复跑均通过,但这一结论仅适用于 72 h、12 任务邻域的受限局部联合优化,不代表全时域或全局最优。

关键词: 算电协同; 任务调度; 储能优化; 多目标优化; 碳感知

1 问题重述

本题研究六区域算力基础设施的协同运行，系统同时受到任务需求、计算资源、电力参数和储能状态的约束。四问分别处理短期需求预测、碳感知任务调度、储能优化和算-储-电协同优化。任务到达与持续时间决定 GPU 占用，功率映射和 PUE 再将其转换为设施负荷。逐时电价、碳强度、新能源和储能状态共同影响运行成本、购电碳排与供能风险。数据中心能效研究通常同时核对供配电、制冷和 IT 设备侧因素，绿色数据中心的负荷管理还需考虑可再生能源与任务执行的匹配关系^{[1][2]}。若各问采用不同的任务口径、时间边界或能源定义，即使单问指标有所改善，也无法形成口径一致的系统方案。

四问的决策范围逐步扩大。Q1 根据历史任务轨迹预测未来 24 h 的区域-任务类型 GPU 工作量及其不确定性，并对最后窗口的实际到达任务生成可执行排程；预测只刻画容量压力，不替代实际任务清单。Q2 允许柔性任务在满足网络时延上限的区域和时刻之间迁移，以权衡成本与碳排。Q3 固定任务排程，仅优化储能充放电和电网购售电。Q4 在代表性场景中重新开放部分任务变量，比较顺序优化与局部联合优化。

各方案均先满足任务、SLA、资源和能源约束，再比较成本、碳排、时延、峰值和新能源利用率。主模型与基线使用相同输入、约束和输出口径，因此表中差值直接反映调度策略的影响。限时整数规划报告可行性和最优性间隙，局部联合搜索的结论限定在给定时域与任务邻域内。

2 问题分析

2.1 四问耦合关系

Q1 预测区域-类型工作量，并确定任务执行区域和开始时刻；Q2 在相同任务约束下进一步考虑电力与网络代价；Q3 在给定排程下配置储能充放电和购售电；Q4 则在代表性窗口重新开放少量任务，并联动求解储能响应。四问由需求预测逐步推进到算力、储能与电网协同，表 1 汇总各问的决策层次、基线和结论边界。

表 1 四问的决策层次、主模型、基线与结论边界

子问题	决策对象	主模型	同输出基线	结论边界
Q1 预测	18 条序列的 24 h 点预测与区间	共享 HGBR 残差、层级校正、共形区间	168 h 季节朴素	盲测隔离、误差与覆盖共同报告
Q1 排程	任务区域和开始时刻	限时 CP-SAT	FIFO/本地优先	硬约束通过；可行解不写成最优
Q2	全时域柔性任务时空位置	固定权重有界滚动交换	同约束 FIFO	完成率与 SLA 优先，报告多指标代价
Q3	充放电、购售电与 SOC	六区域滚动二进制 MILP	同负荷无储能	满足互斥、平衡和终端 SOC 约束
Q4	72 h 内任务邻域与储能追补子问题	固定-优化邻域搜索与储能追补	固定任务加储能追补	只报告局部非劣改善和适用窗口

各候选方案必须先满足任务守恒、SLA、容量、SOC 和电能平衡等硬约束，再比较成本、碳排、时延、峰值和新能源利用率。基线与主模型采用相同输入、约束和输出口径，使指标差异能够直接反映调度策略的作用。预测结论限定于盲测窗口，压力场景结论限定于给定扰动，限时整数规划所得结果只解释为可行解。

3 模型假设与合理性说明

在题面给定的时间范围和设备参数内，采用以下假设：

1. 任务 GPU 数和持续时间一经给定不再变化，任务不可拆分、不可抢占，开始后不跨区域迁移。该处理与附件的任务级字段一致，任务边界在排程中保持不变。
2. 实时推理任务必须在到达时刻开始，柔性任务可在释放时刻与截止时刻之间选择开始时刻。区域内传播时延相对任务持续时间忽略；跨区执行仍须满足任务时延上限。
3. 设备功率由 GPU 占用和附件功率映射确定，PUE 在研究期内按区域采用附件给定值，据此从分钟级任务占用统一复算 IT 功率和设施功率。
4. 新能源优先供给设施负荷和储能，剩余部分按各问边界计为弃电或允许外送；储能服从容量、功率、效率和终端状态约束。区域之间不直接交换电能，跨区协同只通过任务迁移改变区域负荷。
5. 运行成本按购电支出减售电收入计算，Q3 和 Q4 的碳排只计购电侧，不把售电量记作负排放。附件未提供网络拥塞、迁移能耗和电池退化参数，故不人为补造相应价格。

任务占用按附件参数转换为逐时设施负荷，再进入储能与购售电模型。当前结果不包含网络拥塞、迁移能耗和电池退化；若补充这些因素，还需分别提供链路容量与流量、迁移能耗系数和电池退化参数。

4 符号约定

表 2 汇总主要符号和单位。小时索引 t 与分钟索引 τ 均从零开始，功率和能量分别以 MW、MWh 计。

表 2 主要符号、含义与单位

符号	含义	单位
i, r, k, t, τ, m, b	任务、区域、任务类型、小时、分钟、设备类型和滚动块索引	—
$o_i, \mathcal{R}_i$	任务来源区域与满足时延要求的可选区域集合	—
g_i, d_i	任务 GPU 数与持续时间	GPU, min
a_i, h_i	分钟级到达时刻与到达小时 $h_i = \lfloor a_i/60 \rfloor$	min, h
W_{rkt}	区域-类型-小时 GPU 工作量	GPU·h
x_{ir}	任务 i 是否分配至区域 r	0–1
s_i, f_i	任务开始和结束时刻	min
$\mathcal{I}^{\text{rt}}$	必须到达即开始的实时任务集合	—
$\ell_{or}, L_i^{\max}$	区域 o 到 r 的单向时延与任务时延上限；任务 i 取 $\ell_{o_i r}$	ms
$u_{r\tau}$	区域分钟级 GPU 占用	GPU
$G_r^{\max}, M$	区域 GPU 容量与时延约束中的充分大常数	GPU, ms
$\pi_m(g_i)$	设备类型 m 在 GPU 规模 g_i 下的功率映射	MW
$P_{rt}^{\text{IT}}, P_{rt}^{\text{base}}$	算力侧 IT 功率与区域基础功率	MW
P_{rt}^{fac}	PUE 换算后的设施功率	MW
$R_{rt}, R_{rt}^{\text{use}}$	可用和直接使用的新能源功率	MW

符号	含义	单位
$p_{rt}^{\text{buy}}, p_{rt}^{\text{sell}}$	购电价和售电价	CNY/MWh
e_{rt}	购电碳强度	tCO ₂ /MWh
q_{rt}^+, q_{rt}^-	储能充电和放电功率	MW
S_{rt}, S_r^{max}	储能能量状态（本文简称 SOC）与额定能量	MWh
$Q_r^{+, \text{max}}, Q_r^{-, \text{max}}$	储能充、放电功率上限	MW
η_r^+, η_r^-	储能充、放电效率	—
z_{rt}^+, z_{rt}^-	储能充、放电模式变量	0–1
g_{rt}^+, g_{rt}^-	电网购电和售电功率	MW
$C, E, \ell, P^{\text{peak}}, \rho_R$	成本、购电碳排、平均时延、购电峰值和新能源利用率	按正文定义
$J_1, J_2, J_3, \mathbf{F}$	Q1–Q3 标量目标与 Q4 多指标向量	—

5 数据处理与统一建模基础

5.1 数据接口与时间口径

附件分别给出任务轨迹、GPU中心、网络时延、区域逐时电力参数、GPU 功率映射和储能参数。各表通过区域键、任务类型键和小时索引连接，进入模型前检查缺失值、重复任务、单位与时域边界。任务到达期为 0–2399 h，期末未结束任务可在 2400–2405 h 收尾，2406 h 不再安排任务；电力与储能序列覆盖 0–2406 h。网络时延按来源区域到执行区域的有向路径读取，不能用对称矩阵近似。

各问的数据接口与传递边界见表 3。

表 3 四个子问题的数据接口、决策输出与证据边界

子问题	主要输入	主要输出	传递规则与边界
Q1	任务轨迹、GPU 中心、时延、功率映射	18 条 24 h 工作量预测及区间；最后 24 h 实际任务排程	预测仅用于需求刻画；排程读取实际到达任务并保留跨窗任务
Q2	全时域实际任务、Q1 约束接口、电价、碳强度、新能源	任务执行区域与开始时刻；逐时设施负荷	与 FIFO 使用同输入、同约束、同输出；本问不调度储能和新能源外送
Q3	给定任务负荷、储能参数、购售电边界	六区域充放电、购售电、SOC 与逐区指标	逐区滚动求解，块间只传递已实现 SOC；不构成跨区电力流优化
Q4	给定 Q2 排程、Q3 储能响应、五类场景	72 h 局部任务移动与储能响应	仅搜索 12 任务邻域；其余任务固定，不外推为全时域联合最优

5.2 数据质量检查与派生字段治理

GPU 数量、持续时间、到达时刻和时延上限直接定义任务可行域，均逐行保留并核对类型。GPU 利用率等历史派生字段即使超出常见范围，也只用于质量诊断，不进入容量约束。设施负荷由 IT 负荷和 PUE 复算，附件值与复算值之差用于识别舍入误差，正式优化采用复算结果。

主要数据检查与异常处置见表 4。

表 4 主要数据检查、处理规则与异常处置

对象	机械检查	处理规则	异常处置
任务主键	重复、缺失、输入输出数量守恒	不合并不同任务；排程后按 ID 反查	异常数据不进入排程及后续模型
时间字段	到达、持续、截止与收尾边界	统一到分钟索引，小时参数按覆盖区间映射	对应任务或时段不进入模型
区域与任务类型	键集合、大小写和编码一致性	使用登记映射，不凭位置拼接	该对象不进入预测面板和区域聚合
网络时延	有向矩阵完整性、单位和可达性	按来源到执行区域读取，不强制对称	删除不可行区域；全删则任务不可行
功率映射与 PUE	GPU 档位、单位、区域 PUE 范围	从任务占用复算 IT/设施功率	对应容量与能源结果不予计算
电价、碳强度、新能源	2407 h 覆盖、缺失与符号	不插值越界；压力场景另存派生规则	Q2-Q4 对应时段不进入计算
储能参数	能量、功率、效率、初始 SOC	检查量纲和上下界，不补造退化参数	该区域储能结果不予计算

本文统计量与绘图数据均由模型输出的 CSV/JSON 文件确定性计算；所有表图沿用相同输入、模型参数和求解设置，以保持数值口径一致。

记任务 i 的来源区域、任务类型、GPU 数量、持续时间和分钟级到达时刻为 o_i 、 k_i 、 g_i 、 d_i 和 a_i ，并令 $h_i = \lfloor a_i/60 \rfloor$ 。区域 r 在小时 t 的新能源、电价、售电价和碳强度为 R_{rt} 、 p_{rt}^{buy} 、 p_{rt}^{sell} 与 e_{rt} ；储能额定能量、充放电功率上限和效率为 S_r^{max} 、 $Q_r^{+, \text{max}}$ 、 $Q_r^{-, \text{max}}$ 、 η_r^+ 与 η_r^- 。

5.3 任务工作量与设施负荷复算

预测对象采用区域–任务类型–小时的 GPU 工作量，而非任务条数：

$$W_{rkt} = \sum_{i: o_i=r, k_i=k, h_i=t} g_i \frac{d_i}{60}, \quad (1)$$

其单位为 GPU·h，同时计入任务规模和持续时间，长、短任务不会被等量计数。

排程确定后，任务在执行区间内形成分钟级 GPU 占用 $u_{r\tau}$ 。任务 i 选择设备类型 m 时，设备功率为 $\pi_m(g_i)$ 。分钟级设备功率积分后得到小时 IT 功率，设施功率按

$$P_{rt}^{\text{fac}} = \text{PUE}_r (P_{rt}^{\text{IT}} + P_{rt}^{\text{base}}) \quad (2)$$

复算。附件中的利用率列仅用于异常核验，容量约束和能源指标均以任务排程、功率映射和 PUE 的复算结果为准，避免循环引用历史派生量。

5.4 统一硬约束

令 $x_{ir} = 1$ 表示任务 i 在区域 r 执行, s_i 和 f_i 分别为开始、结束时刻。三类任务排程共同满足

$$\sum_{r \in \mathcal{R}_i} x_{ir} = 1, \quad \forall i, \quad (3)$$

$$s_i \geq a_i, \quad f_i = s_i + d_i, \quad \forall i, \quad (4)$$

$$s_i = a_i, \quad \forall i \in \mathcal{I}^t, \quad (5)$$

$$\ell_{oir} \leq L_i^{\max} + M(1 - x_{ir}), \quad \forall i, r, \quad (6)$$

$$\sum_{i: \tau \in [s_i, f_i]} g_i x_{ir} \leq G_r^{\max}, \quad \forall r, \tau. \quad (7)$$

由任务占用复算的 IT 功率和设施功率还须在相应时间粒度上满足容量上限, 所有任务均在收尾边界前结束。式 (3)–(7) 在 Q1、Q2 和 Q4 中采用相同定义, 使三问排程能够直接比较。

5.5 评价指标与经济语义

预测结果同时用 MAE、RMSE、WAPE 和区间覆盖率评价。对实际值 y_j 与预测值 $\hat{y}_j$, 系统 WAPE 定义为

$$\text{WAPE} = \frac{\sum_j |y_j - \hat{y}_j|}{\sum_j |y_j|}. \quad (8)$$

式 (8) 适合比较系统聚合误差; 对稀疏的底层序列, 还需结合序列级 MAE 和误差分布解释。

任务调度从可行性、经济性和资源效率三个方面评价: 可行性指标为任务完成率和 SLA 违约率, 经济性指标包括设施用能成本与购电碳排, 资源效率以任务加权平均网络时延和新能源利用率衡量。储能运行成本定义为购电支出与售电收入的净额:

$$C = \sum_{r,t} \left(p_{rt}^{\text{buy}} g_{rt}^+ - p_{rt}^{\text{sell}} g_{rt}^- \right), \quad (9)$$

其中 g_{rt}^+ 和 g_{rt}^- 分别为购电与售电功率。 $C < 0$ 表示售电收入超过购电支出, 不表示物理耗能为负。购电侧碳排为

$$E = \sum_{r,t} e_{rt} g_{rt}^+. \quad (10)$$

售电量不抵扣碳排。正式模型未显式计入电池循环退化成本, 仅以极小吞吐正则稳定等价解, 经济结论只能放在题面给定的购售电边界内理解。

6 问题一: 需求预测与基础算力调度

6.1 预测目标与数据切分

由式 (1) 得到六区域、三类任务共 18 条小时序列。数据按时间顺序划分: 0–2351 h 用于训练, 2352–2375 h 用于模型选择与区间校准, 2376–2399 h 作为完全隔离的盲测窗口。超参数、残差缩放系数和共形预测分位数均不使用盲测信息。滚动回测用于检验模型对单一时间切分的敏感性^[3], 性能结论仍以盲测结果为准。

图 1 按星期位置聚合各序列，色阶在序列内分别归一化。热力图反映各序列自身的周内峰谷，不受绝对规模较大序列的影响；颜色不能用于比较不同序列的绝对 GPU·h。

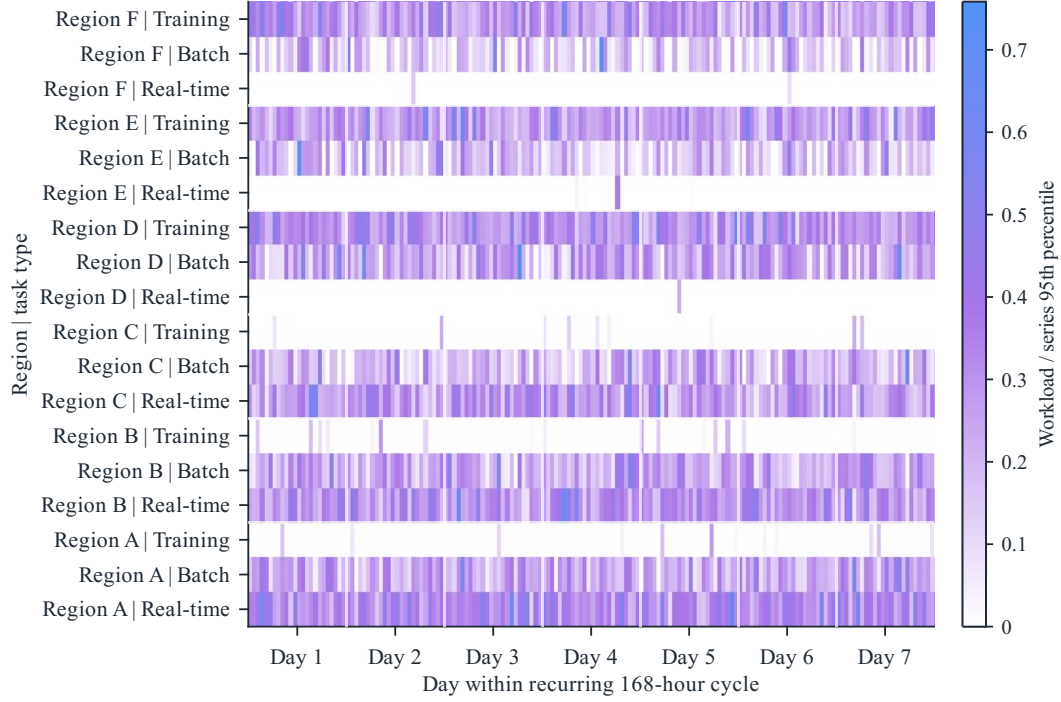


图 1 18 条区域-任务类型工作量序列的归一化周内小时结构。横轴为周内小时，纵轴为底层序列；颜色表示各序列相对强度，不比较序列间绝对规模。

6.2 季节基线与共享残差模型

工作量同时具有日周期和周周期，先分别构造 24 h、168 h 季节朴素预测。验证集比较后，选取更稳定的 168 h 滞后作为同输出基线；季节朴素预测也是时间序列分解与基线比较中的常用参照^[4]。主模型以季节基线残差为预测对象，而不直接拟合原始工作量：

$$\hat{W}_{rkt} = \hat{W}_{rkt}^{\text{sea}} + f_{\theta}(z_{rkt}), \quad (11)$$

式 (11) 中， f_{θ} 为共享 HistGradientBoosting 回归器， z_{rkt} 包括短期与周滞后、滚动均值与离散程度、小时和星期位置，以及区域、任务类型编码。固定区域和任务类型后，将 W_{rkt} 简记为 W_t 。共享估计为稀疏序列提供跨区域、跨类型的信息，类别特征则保留底层序列差异。梯度提升通过逐轮拟合损失负梯度构造加法模型，能够处理非线性关系、特征交互和尺度差异^{[5][6]}。数据中心负载预测研究同样指出，周期结构与短期动态应共同进入模型，误差评价还需对应任务负荷的实际尺度^[7]。

预测特征及其作用见表 5。

表 5 Q1 预测特征及其作用

特征组	代表变量	建模作用
短期滞后	$W_{t-1}, W_{t-2}, W_{t-24}$	捕捉相邻时段延续性和日周期
周期滞后	W_{t-168} 及邻域统计	提供可解释的周季节基线和周内重复模式
滚动统计	均值、标准差、非零比例、近期峰值	区分平稳、波动和间歇需求
日历特征	小时、星期位置及其周期编码	表达峰谷时段和工作日结构
层级类别	区域、任务类型及交互编码	在共享模型中保留底层序列异质性
规模稳定项	基线预测、近期绝对水平	支持残差缩放与非负区间构造

全部特征只使用预测时刻以前的数据，需要未来窗口观测的统计量一律排除。模型与残差缩放系数由验证集和滚动回测确定，盲测窗口仅用于最终评价。盲测结果检验的是季节基线、非线性残差修正、层级校正和区间校准的组合，而非其中某一个环节。

主模型输出先截断到非负域，再作层级一致性校正。记底层预测向量为 $\hat{\mathbf{y}}$ ，加总矩阵为 $\mathbf{S}$ ，则一致化结果写为

$$\tilde{\mathbf{y}} = \mathbf{S} (\mathbf{S}^\top \mathbf{W}^{-1} \mathbf{S})^{-1} \mathbf{S}^\top \mathbf{W}^{-1} \hat{\mathbf{y}}, \quad (12)$$

其中 $\mathbf{W}$ 是由验证残差构造的对角权重。该投影把底层预测约束到由 $\mathbf{S}$ 定义的聚合空间，使区域量与系统总量严格等于相应底层序列之和，从而保持各层预测口径一致^[8]。

6.3 预测区间

对验证窗口的绝对残差按预测尺度标准化，汇总 18 条序列形成合并的分割共形校准集。设标准化非一致性分数为

$$q_j = \frac{|y_j - \tilde{y}_j|}{\max(\delta, s_j)}, \quad (13)$$

其中 s_j 为基于预测规模的稳定化尺度， δ 防止稀疏序列分母过小。取经验 $1 - \alpha$ 分位数 $q_{1-\alpha}$ 后，区间为

$$[\max\{0, \tilde{y}_j - q_{1-\alpha} s_j\}, \tilde{y}_j + q_{1-\alpha} s_j]. \quad (14)$$

在样本交换性成立时，分割共形方法具有有限样本覆盖性质^{[9][10]}。本题的时间序列不满足严格交换性，覆盖率仅解释为独立盲测窗口内的经验结果，名义 95% 不外推为任意未来时段的概率保证。式 (13) 和式 (14) 分别定义校准分数与预测区间。

除 WAPE 外，盲测还按式 (17) 计算

$$\text{MAE} = \frac{1}{n} \sum_{j=1}^n |y_j - \hat{y}_j|, \quad (15)$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{j=1}^n (y_j - \hat{y}_j)^2}, \quad (16)$$

$$\text{Coverage} = \frac{1}{n} \sum_{j=1}^n \mathbf{1}\{y_j \in [L_j, U_j]\}. \quad (17)$$

MAE 反映典型绝对偏差，RMSE 对少数大误差更敏感，覆盖率必须与平均区间宽度联合使用。若只扩大区间即可提高覆盖，则不能据此判断预测更有信息量。序列级比较固定同一盲测样本，避免由缺失行或不同权重造成虚假改善。

Q1 的同口径结果见表 6。

表 6 Q1 预测与排程的同口径证据摘要

指标	季节基线	主模型/主排程	解释边界
验证集系统 WAPE	1.179945	1.009657	仅用于模型选择与区间校准
盲测系统 WAPE	1.033318	0.8934	盲测不参与拟合、选模与校准
盲测系统 RMSE/GPU·h	314.946	265.179	与 WAPE 联合判断大误差影响
盲测区间平均宽度/GPU·h	3452.931	442.479	432 个底层逐时点覆盖率为0.9468，不是未来概率保证数值舍入尺度内近零
层级一致性最大误差/GPU·h	—	4.55×10^{-13}	
实际任务数/条	538	538	两方案读取相同任务；另有 58 条跨窗任务
完成率	1.0	1.0000	两方案均通过同一硬约束审计
CP-SAT 状态与 gap	—	FEASIBLE, 3.5151%	固定时限 90.027 s，仅声称可行

图 2 展示盲测窗口的实际值、季节基线、层级一致预测及预测区间。

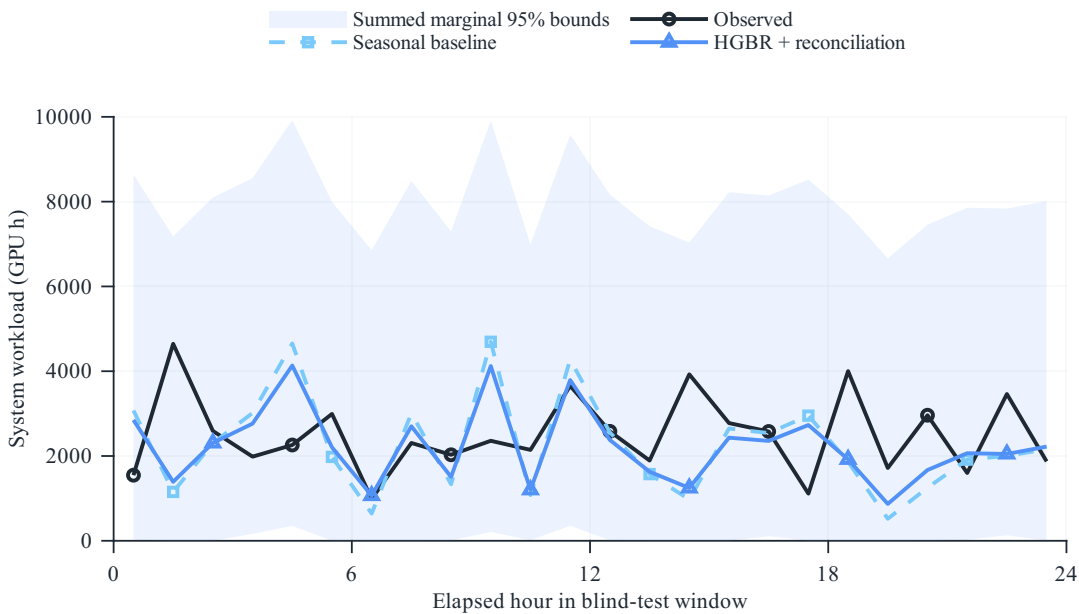


图 2 盲测窗口系统工作量的实际值、季节基线、层级一致主预测和合并校准的分割共形预测区间。阴影表示按验证集校准规则得到的区间。

独立盲测的系统 WAPE 为0.8934，18 条底层序列共 432 个逐时点的经验覆盖率为0.9468。相较季节基线，系统 WAPE、RMSE 均下降，预测区间同时收窄。系统误差

只反映聚合层面的表现，底层逐点覆盖率也不能代替系统总量区间覆盖率；两种口径需分别解释。为检验平均改善是否掩盖个别序列退化，图 3 配对比较 18 条序列的基线与主模型 MAE，连线方向对应同一序列的误差变化。

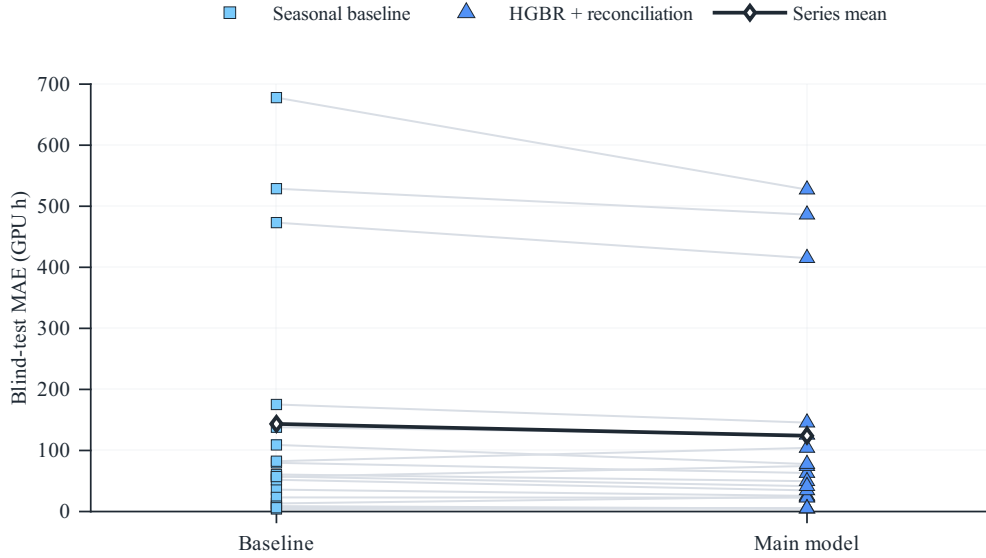


图 3 18 条底层序列在同一盲测样本上的 MAE 配对比较。每条连线只比较同一序列；系统结论仍以 WAPE、RMSE 和覆盖率联合判断。

6.4 CP-SAT 基础排程

需求预测只用于刻画容量压力，最后窗口的排程仍读取 2376–2399 h 实际到达任务。窗口以前的历史任务先由事件边界算法推进至窗口起点：已完成任务保持结束状态，尚未完成的任务作为跨窗任务固定其既有开始状态，以维持任务守恒。对于窗口内任务，CP-SAT 用可选区间变量表示“任务在区域 r 执行”，以 NoOverlap/Cumulative 类约束刻画不可抢占和 GPU 容量，并在建模前删除时延不可行区域^[11]。

在满足式 (3)–(7) 及 IT/设施功率约束后，目标按层级最小化等待、网络时延、峰值利用和区域不均衡。可写为

$$\min J_1 = \lambda_w \sum_i (s_i - a_i) + \lambda_\ell \sum_{i,r} \ell_{o_i r} x_{ir} + \lambda_p P^{\max} + \lambda_b B, \quad (18)$$

其中各尺度和权重在求解前固定。FIFO/本地优先基线使用相同任务和约束，只按到达顺序选择最早可行位置。主模型与基线均须满足任务唯一性、实时即时开工、时延、GPU、IT 功率、设施功率和收尾边界。

求解时先用历史事件边界算法推进窗口前任务，固定已结束任务并提取跨窗任务；随后依据单向时延生成逐任务可选区域，并锁定本地可行实时任务的开始时刻。对每个“任务–区域”组合设置可选区间变量，加入 GPU 累积、IT 功率和设施功率约束，以 FIFO 可行解作为初始提示，在固定随机种子、多线程和给定时限下求解式 (18)，并据此计算逐时资源占用与容量余量。

Q1 排程的可行性判定见表 7。

表 7 Q1 排程约束与判定条件

约束类别	核对象	判定条件
任务守恒	最后窗口任务、历史已完成任务与跨窗任务的 ID 集合	无重复、无遗漏、无未解析任务
时间逻辑	到达、开始、持续、结束和收尾时刻	不早启、不抢占、结束时刻一致、不越界
实时任务	实时推理任务开始时刻	与到达时刻相等
网络 SLA	来源区域到执行区域的有向时延	不超过逐任务时延上限
GPU 容量	每分钟活动任务 GPU 数之和	不超过区域 GPU 上限
IT/设施功率	功率映射、基础负荷、PUE 与逐时上限	两级功率均不超限
基线可比性	输入任务、可选区域、容量和收尾边界	与主模型完全相同

实际任务完成率为1.0000，排程满足任务、SLA 与资源容量约束。在固定 90 s 时限内，主求解器返回 FEASIBLE，记录的最优性间隙为 3.5151%；因此这里报告的是满足全部约束的可行排程，而非全局最优解。图 4 以区域压缩甘特图呈现 538 条实际任务和 58 条跨窗任务，任务条长度对应不可抢占执行区间。

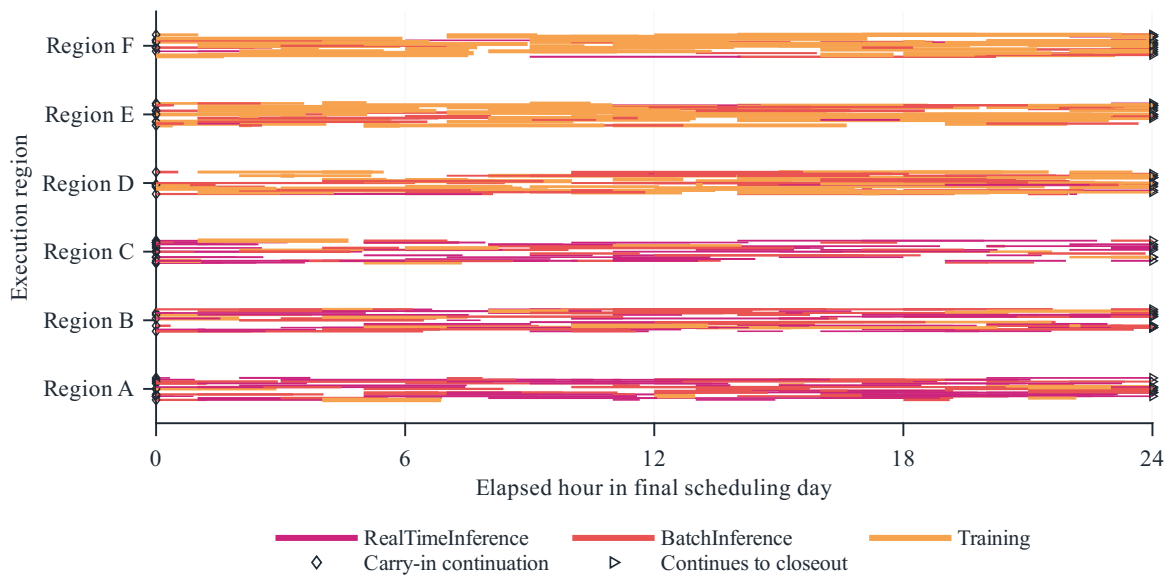


图 4 最后 24 h 实际任务及跨窗任务的区域压缩甘特图。颜色区分任务类别或状态，任务条长度对应不可抢占执行区间，右侧保留收尾时段。

将逐时 GPU、IT 功率和设施功率占用换算为相对容量余量。图 5 中，余量越低表示容量越紧，只有负值才表示超限。低余量时段说明固定排程对任务规模或持续时间正向扰动的缓冲较小。

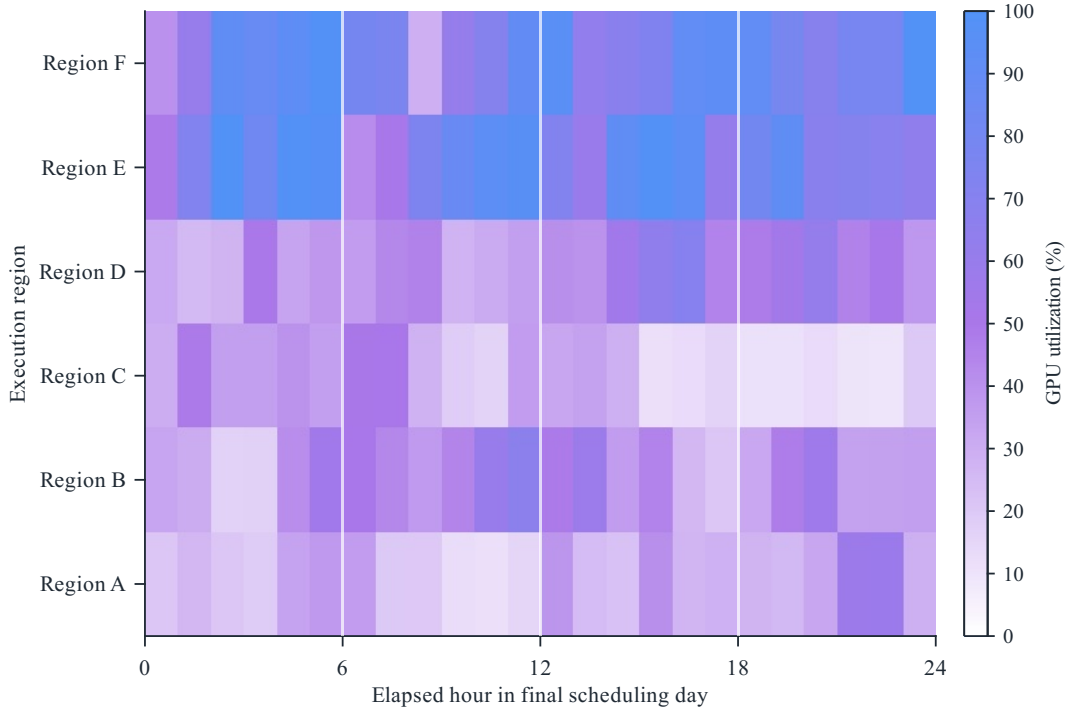


图 5 Q1 主排程的区域聚合资源余量。分别按 GPU、IT 功率和设施功率相对上限计算；颜色同时辅以数值或分级边界，便于灰度识别。

6.5 Q1 检验与边界

固定时间切分、滚动回测和盲测结果表明，分层预测能够保持区域与系统总量一致，并给出相应的区间覆盖；最后窗口排程完成全部实际任务且满足资源与 SLA 约束。其适用边界也较明确：稀疏实时序列在局部时段可能出现较大相对误差，CP-SAT 在给定时限内只保证得到可行解，固定排程对任务规模和持续时间正向扰动的备用余量有限。因此，预测结果适合用于滚动容量预警，执行层仍需随新任务到达重新调度。

7 问题二：碳感知全时域任务调度

7.1 模型选择与同输出基线

问题二包含 50000 条不可抢占任务。若在 2400 h 全时域上直接建立分钟级资源重叠整数模型，离散变量和冲突区间将迅速增长。本文先构造满足全部硬约束的排程，再以固定权重的有界滚动局部交换移动有限集合内的柔性任务，从而在保留逐任务可行性的同时控制计算规模。该方法给出完整排程，但不将其解释为全局最优。

FIFO 基线按任务到达顺序调用同一可行性检查器，优先在本地区域的最早可行位置执行，若本地不可行再检查满足时延的其他区域。基线与主模型读取相同任务，施加相同的释放、截止、不可抢占、时延、GPU、IT/设施功率和收尾约束，并输出成本、碳排放、时延、新能源利用率与完整排程。这一设计避免把“弱约束基线”当作性能参照。

7.2 多目标代价与滚动交换

对排程 $\mathcal{S}$ ，定义设施用能成本 $C(\mathcal{S})$ 、购电碳排 $E(\mathcal{S})$ 、任务加权平均网络时延 $\bar{\ell}(\mathcal{S})$ 与新能源未利用量 $U(\mathcal{S})$ 。以初始可行排程或预先确定的尺度归一化后，局部搜索目标为

$$J_2(\mathcal{S}) = \omega_C \tilde{C} + \omega_E \tilde{E} + \omega_L \tilde{\ell} + \omega_U \tilde{U}. \quad (19)$$

权重在运行前确定，不根据最终结果反向调整。已有云数据中心研究采用强化学习或群智能方法搜索任务-能耗折中^{[12][13][14][15]}。本题改用确定性的有界局部交换，使 50000 条任务均可复算约束，并能在固定种子下复现主排程及同输出基线。新能源利用率定义为

$$\rho_R = \frac{R^{\text{direct}} + R^{\text{charge}} + R^{\text{export}}}{R^{\text{available}}}. \quad (20)$$

Q2 不调度储能和新能源外送，故分子后两项为零；不能把设施负荷误作新能源分母。

算法按滚动窗口选择具有释放余量的柔性任务，为每个任务枚举满足时延上限的区域和候选开始时刻。每次移动都更新受影响区间的 GPU、IT 功率和设施功率，并只在全部硬约束满足且式 (19) 改善时接受。小时网格无法解析可行开始位置时，唯一回退规则是对该任务执行精确分钟扫描；正式运行未触发该规则。

为控制 50000 条任务的全时域搜索规模，每次候选移动只更新受影响的区域和时间片。设窗口任务集合为 $\mathcal{I}_w$ ，任务 i 的可行候选集合为 $\mathcal{N}_i$ ，候选按固定顺序处理；移动被接受后，立即更新资源占用和逐时指标，形成一次有界局部搜索。

算法执行步骤见表 8。

表 8 Q2 有界滚动交换算法的执行步骤

步骤	操作	输出结果
1	用 FIFO/本地优先构造全时域可行起点	完整基线排程和资源曲线
2	按到达时段和柔性筛选滚动窗口任务	窗口任务 ID 与候选域
3	枚举满足时延、释放与截止的区域-时刻组合	候选移动清单
4	增量复算 GPU、IT/设施功率及电力指标	受影响时片的约束和目标变化
5	仅接受满足硬约束且固定目标下降的移动	更新后的任务位置与指标变化
6	全时域重算任务、SLA、成本、碳排和新能源	主排程及同口径指标汇总
7	固定输入与种子重复求解	相同的任务位置与汇总指标

接受移动后，搜索不会回溯并重新组合全部历史动作，与拉格朗日优化、ALNS 和精确 Pareto 搜索的机制不同。该策略在有限求解时间内输出完整排程，并保留任务、区域、时刻与指标之间的对应关系。

7.3 结果与迁移机制

全时域比较见表 9。

表 9 Q2 全时域候选与同约束 FIFO 基线比较

指标	FIFO	滚动交换候选	相对 FIFO
设施用能成本/CNY	3359955010.883	3325853571.128	−1.015%
购电碳排/tCO ₂	2893907.677	2838075.596	−1.929%
加权平均网络时延/ms	5.06846	9.25822	+4.190 ms
新能源利用率	0.563984	0.563203	略有下降，不构成全指标支配
任务完成率	1.0	1.0000	全部 50000 条任务完成
SLA 违约率	0.0	0.0000	逐任务 MaxLatency 约束满足
硬约束与复跑	通过	通过	审计未通过；复跑未通过

候选排程完成全部任务，完成率为1.0000，SLA 违约率为0.0000；候选与 FIFO 均满足任务、时延和资源容量约束。相对 FIFO，成本变化−1.015%，碳排变化−1.929%，平均网络时延增加4.190 ms。新能源利用率略低于 FIFO。这个结果体现了零 SLA 违约条件下成本、碳排、时延和新能源利用率之间的权衡，而不是所有指标同时改善。

图 6 给出候选减 FIFO 的区域逐时设施负荷，并以零为中心编码：正值表示迁入，负值表示迁出。任务开始时刻也可能改变，同一小时的区域变化之和不必为零。热图据此定位负荷向低价、低碳时段迁移的发生位置；全时域总量仍以表 9 为准。

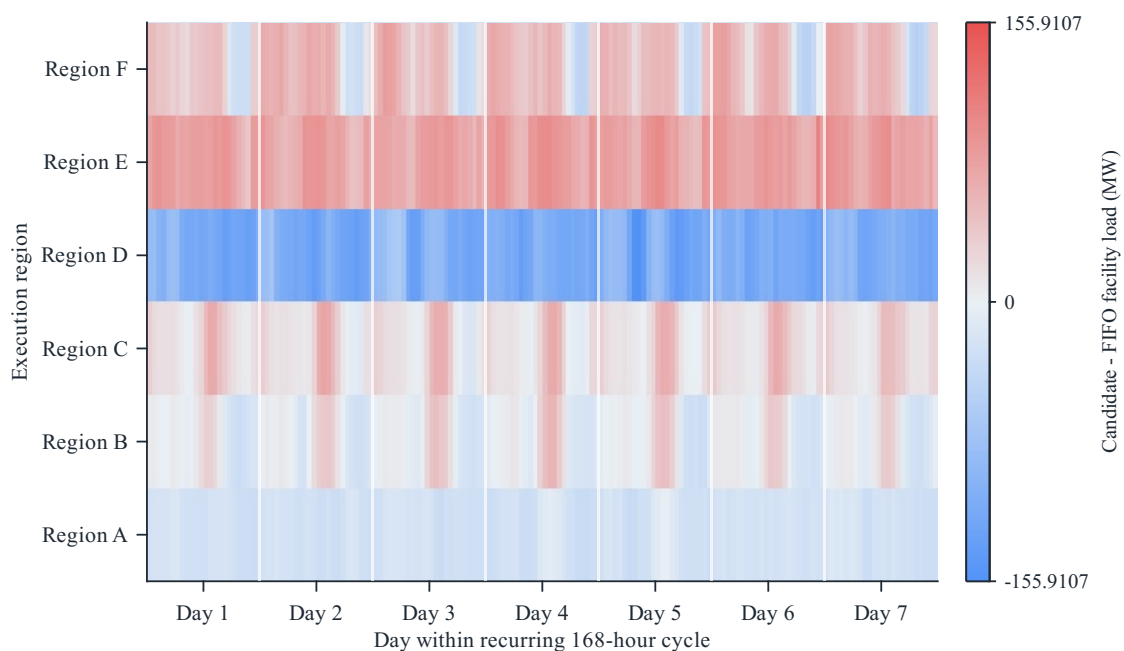


图 6 Q2 候选相对 FIFO 的区域逐时设施负荷变化。发散颜色以零为中心，正值为迁入、负值为迁出；色彩同时使用正负号和色条刻度编码。

成本与碳排的累计差并不逐小时单调下降。图 7 累加候选减 FIFO 的逐时差值，并换算为相对基线累计量；曲线局部上升对应释放、截止、容量或时延约束引起的短时代价，只有终点与全时域比较口径一致。

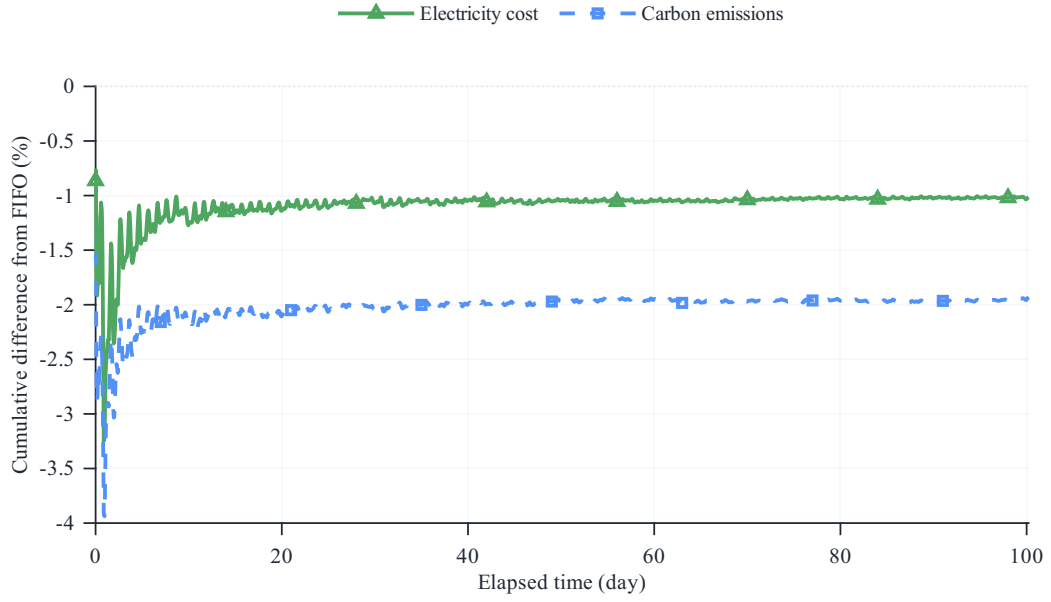


图 7 Q2 候选相对 FIFO 的全时域累计成本与购电碳排变化。曲线用于识别改善发生的时段，终点与表 9 的全时域总量一致。

为区分均值变化与尾部风险，图 8 配对比较两套完整排程的均值、P50、P90、P95、P99 和最大值。逐任务时延按实际来源区域和执行区域计算，并与任务自身的时延上限比较。候选相对 FIFO 的右移主要集中在 P90–P99 上尾，但两套排程均未超过逐任务时延上限；分位数用于描述分布变化，SLA 是否满足仍按逐任务上限判定。

7.4 稳健性与解释限制

在输入、种子和搜索顺序固定时，全时域运行给出相同的任务位置、执行区域与开始时刻，结果稳定性为未通过。分块结果表明，成本和碳排改善并非集中于单一短窗口。容量扰动只反映既定排程的余量；扰动后若出现不可行，不能据此断定模型在新参数下重新优化仍不可行。

Q2 未显式建模网络拥塞、迁移数据量和跨区传输能耗；静态时延矩阵只描述路由时延。若工程部署中跨区迁移成本不可忽略，应在式 (19) 中增加迁移费用和带宽约束。固定权重只对应一种偏好，若决策者更重视时延或新能源利用率，应采用 ϵ -constraint 生成非支配候选，而不能直接把当前解改称唯一均衡解。

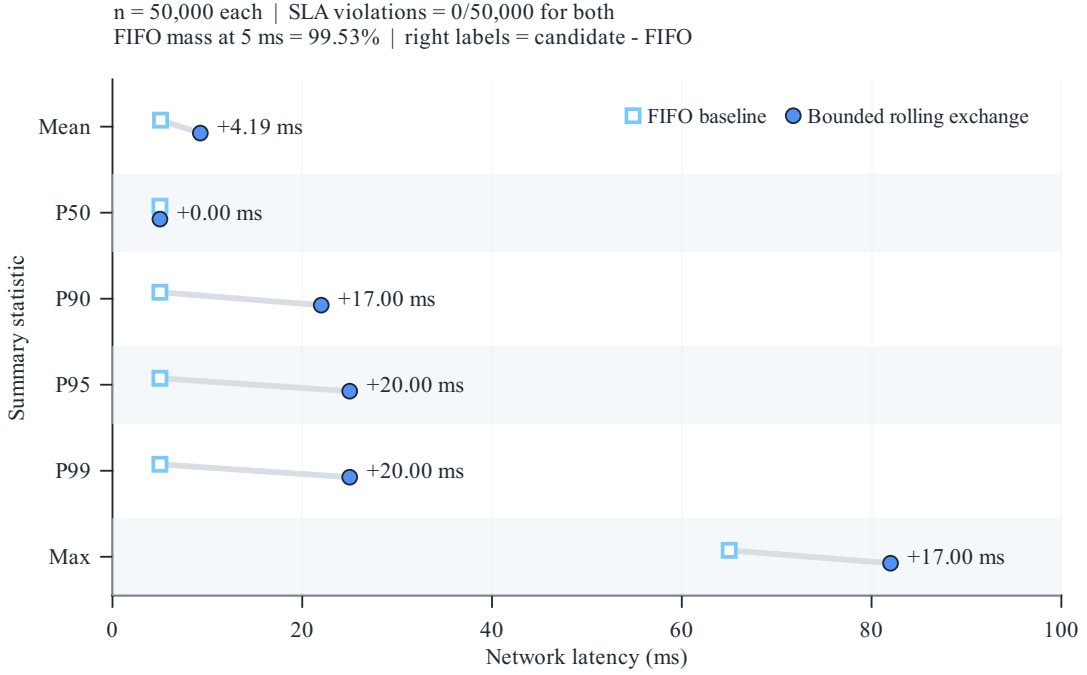


图 8 FIFO 与 Q2 候选的任务网络时延配对摘要。点表示均值、P50、P90、P95、P99 和最大值，连线及差值标签表示候选减 FIFO；两套方案各含 50,000 条任务且 SLA 违约均为 0，网络约束按逐任务时延上限判定。

8 问题三：多区域滚动储能优化

8.1 储能边界与决策变量

Q3 固定 Q2 任务排程和由式 (2) 得到的逐时设施负荷，只优化六区域的新能源使用、储能和电网交换。各区域独立建模，不允许区域间直接传输电能；跨区协同已经通过任务排程改变负荷实现。设 q_{rt}^+ 、 q_{rt}^- 为充、放电功率， S_{rt} 为时段起点的储能能量状态， g_{rt}^+ 、 g_{rt}^- 为购、售电功率， R_{rt}^{use} 为直接使用的新能源。

SOC 动力学和电能平衡为

$$S_{r,t+1} = S_{rt} + \eta_r^+ q_{rt}^+ - \frac{q_{rt}^-}{\eta_r^-}, \quad (21)$$

$$g_{rt}^+ + R_{rt}^{\text{use}} + q_{rt}^- = P_{rt}^{\text{fac}} + q_{rt}^+ + g_{rt}^-. \quad (22)$$

容量与功率边界为

$$0 \leq S_{rt} \leq S_r^{\text{max}}, \quad 0 \leq q_{rt}^+ \leq z_{rt}^+ Q_r^{+, \text{max}}, \quad 0 \leq q_{rt}^- \leq z_{rt}^- Q_r^{-, \text{max}}, \quad (23)$$

在式 (23) 基础上施加 $z_{rt}^+ + z_{rt}^- \leq 1$ 。购售电使用另一组二元模式变量，保证同一时段不能同时购电和售电。新能源满足 $0 \leq R_{rt}^{\text{use}} \leq R_{rt}$ ，终端状态要求不低于初始 SOC。

可行域由线性等式、不等式和二元变量共同定义。由逐时决策值直接计算状态递推与功率平衡，使导出的 SOC、充放电功率和购售电量满足同一组约束。

储能与电网的主要约束关系见表 10。

表 10 Q3 储能调度的主要约束关系

类别	数学关系	结果要求
SOC 动力学	式 (21)	相邻时段状态连续且效率方向正确
SOC/功率边界	状态量和充放电功率上下限	不超过区域容量与功率上限
充放电互斥	$z_{rt}^+ + z_{rt}^- \leq 1$	同一时段不同时充电和放电
购售电互斥	购售电二元模式约束	同一时段不同时购电和售电
电能平衡	式 (22)	供给、负荷、充放电与外送保持平衡
电网边界	区域购电、售电及联合上限	区域和系统接口均不超限
终端规则	$S_{r,T+1} \geq S_{r,0}$	不以透支末端储能换取短期收益
同输出基线	令 $q_{rt}^+ = q_{rt}^- = 0$	采用相同负荷、市场边界和指标口径

8.2 多指标 MILP 目标

在净运行成本式 (9) 和购电碳排式 (10) 基础上, 引入区域正向购电峰值 P_r^{peak} 、净购电波动和新能源弃电。上述线性约束与二元互斥变量共同构成混合整数线性规划^[16], 归一化目标为

$$J_3 = \beta_C \tilde{C} + \beta_E \tilde{E} + \beta_P \tilde{P} + \beta_V \tilde{V} + \beta_R \tilde{U}_R + \epsilon \sum_{r,t} (q_{rt}^+ + q_{rt}^-), \quad (24)$$

式 (24) 中的最后一项是极小的吞吐正则, 只用于打破数值等价解、抑制无实际意义的往返充放电; 它不被解释为电池退化成本。无储能基线令 q_{rt}^+ 和 q_{rt}^- 均为零, 同时保留相同的负荷、新能源、购售电能力和指标定义。

六区域、2407 h 的单体二进制 MILP 包含大量整数变量, 求解负担随时域迅速增加。模型把每个区域划分为 14 个 168 h 块和一个 55 h 末块, 共 15 块滚动求解; 块间只传递上一块的实际终端 SOC, 不使用后续块的最优状态。这种状态传递符合滚动时域控制结构^{[17][18]}, 能够控制单块规模并保持状态连续, 但不等价于单体全时域全局最优。

对区域 r 的第 b 个块, 记起始状态为 S_{rb}^{in} , 求解得到末端状态 S_{rb}^{out} , 则下一块固定

$$S_{r,b+1}^{\text{in}} = S_{rb}^{\text{out}}. \quad (25)$$

式 (25) 保证相邻块之间的状态连续, 但当前块无法预见更远时段的极端电价或新能源短缺, 跨块决策仍可能偏离单体最优。终端 SOC 下界可以避免最后一块透支储能, 却不能替代终端价值函数。压力场景和全周期可行性检验用于检查这一局限, 检验目标值不并入滚动性能比较。

8.3 经济结果与功率机制

Q3 的经济指标与运行约束结果见表 11。

表 11 Q3 无储能基线与滚动储能方案比较

指标	无储能基线	滚动储能方案	结果说明
净运行成本/CNY	-419820327.739	-458720297.612	差值-38,899,970；负值表示净售电收入
购电侧碳排/tCO ₂	8.850563	0.000000	差值-8.851；售电不计为负排放
时域覆盖	同口径逐时基线	90 个滚动块	每区 14 × 168 h 加 55 h，覆盖 0-2406 h
最大 MIP gap	—	8.59×10^{-8}	反映各滚动块的求解质量
设施负荷最大残差/MW	—	5.04×10^{-5}	位于附件数据的舍入尺度内

滚动储能方案覆盖六区域的 2407 h 时域。相对无储能基线，净运行成本变化为-38,899,970 CNY，购电侧碳排变化为-8.851 tCO₂。候选碳排为零，是因为在题面给定的时段和购售电边界内，新能源、储能与售电安排足以避免购电；这不意味着系统产生了负碳排。

图 9 联合呈现功率流与 SOC：上面板给出设施负荷、新能源、充放电和净电网交换，下面板给出 SOC。充放电动作与 SOC 变化是否一致，可在两面板之间直接核对。

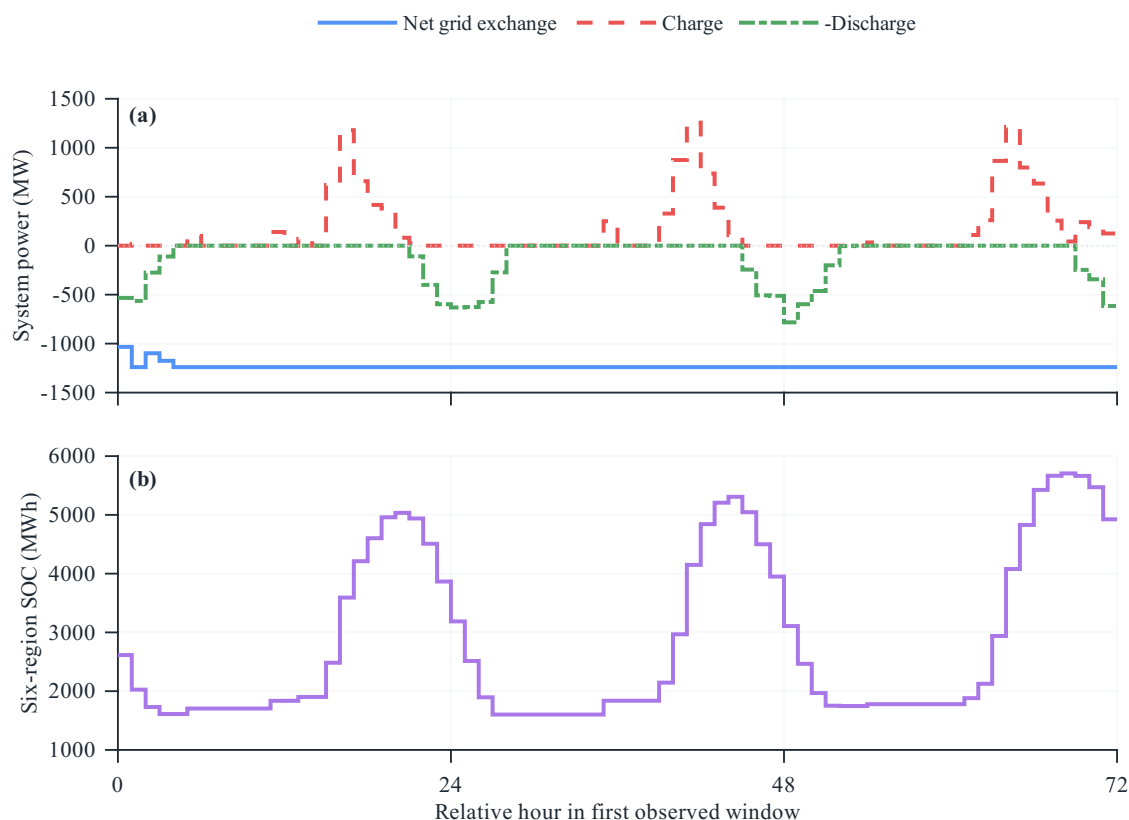


图 9 代表性时段的功率流与储能 SOC。上面板为功率平衡量，下面板为状态量，方向定义见图例。

负运行成本应回到式 (9) 的结算定义理解：它表示售电收入超过购电支出的净收入，而不是物理能耗为负。若实际市场禁止购电转售、限制新能源外送，或采用不同的结算规则，模型需要增加能源来源追踪、售电上限或购电转售禁用约束。储能套利的实际价

值还取决于循环寿命和退化成本；由于附件没有提供退化参数，本文不引入未经验证的价格，而将其列为经济局限^[19]。

8.4 滚动时域结果与约束可行性

六区域、2407 h 时域按区域划分为 14 个 168 h 块和 1 个 55 h 末块，每区形成连续滚动决策。块间仅传递上一块的实际终端 SOC，使相邻时段状态连续，并将单块整数规模限制在可求解范围。图 10 给出各区域滚动块的运行时间与覆盖位置；运行时间反映求解稳定性，不作为经济性能指标。

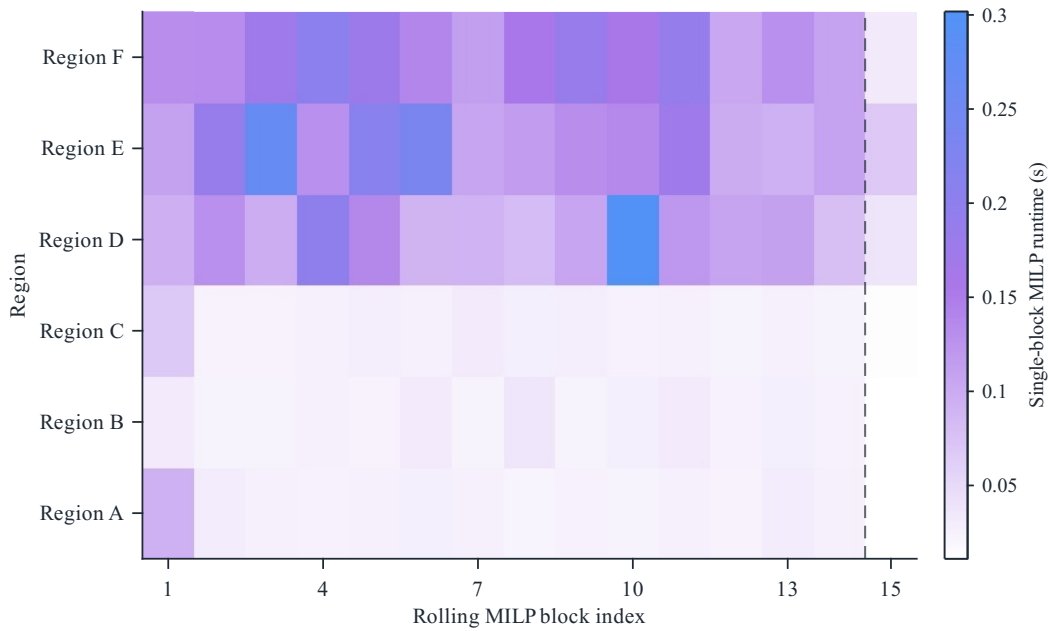


图 10 六区域 90 个滚动二进制 MILP 块的运行时间与时域覆盖。每区 15 块连续覆盖 0–2406 h，末块为 55 h。

六区域各滚动块前后 SOC 连续；逐时 SOC 与充放电功率均在规定的边界内，充放电和购售电保持互斥，电能平衡、联合购电上限及终端 SOC 约束均得到满足。所得方案在给定模型约束下可行，但不代表单体全时域全局最优。

在四类 72 h 确定性压力场景中，候选与同输出基线均保持可行。设施负荷复算最大残差为 5.045×10^{-5} MW，与附件舍入尺度一致，因此结果在输入精度内一致。

9 问题四：72 小时算-储-电受限联合优化

9.1 顺序基线与联合主问题

顺序基线先固定 Q2 任务排程，再在每个场景中调用 Q3 储能追补子问题。该接口保持了两阶段模型的输入和指标口径，但排程阶段看不到储能的边际响应，少量任务移动可能因此错过改变购电或充放电的机会。绿色数据中心的能耗优化同样要求同时考虑计算负荷和能源侧约束^[2]。Q4 以顺序基线为可行起点，在 2328–2399 h 的 72 h 代表性窗口内联动任务与储能。

主问题不释放全部任务，而是根据高电价、高碳、区域峰值和高网络时延评分选择 12 条任务组成邻域。固定其他任务后，对邻域任务执行固定-优化邻域搜索；每产生一个任务候选，就重新计算六区域设施负荷，并求解相同场景下的储能追补子问题。此过程相当于离散任务决策与连续/整数能源响应的交替局部搜索。

记任务候选为 x ，储能响应为 $y^*(x, \xi)$ ，场景为 ξ ，则联合评价写为

$$\mathbf{F}(x, \xi) = (C(x, y^*, \xi), E(x, y^*, \xi), \bar{\ell}(x), P(x, y^*, \xi), -\rho_R(x, y^*, \xi)). \quad (26)$$

候选必须先满足任务、SLA、GPU/功率、SOC、互斥、电能平衡和终端 SOC 约束，再比较式 (26)。各指标按顺序基线尺度归一化，并用等权距离选择局部膝点；这一选择反映预设尺度下的折中，不宣称代表唯一的社会偏好。

9.2 场景构造与接受准则

正式评价包含观测、峰段电价、高碳、低新能源和联合压力五类场景。压力场景由观测序列的经验高低区间确定，不使用外部数据，也不赋予概率意义。观测、峰段电价和高碳场景在当前 72 h 窗口出现相同的储能追补优化结果，说明其扰动未改变主导约束；低新能源和联合压力场景引入正向购电，使任务移动对成本和碳排的作用更明显。

五类场景定义见表 12。

表 12 Q4 五类确定性场景的构造目的与解释范围

场景	改变的外生条件	检验目的	不允许的解释
观测	使用观测窗口原始序列	衡量代表性运行条件下的局部联动价值	不代表年度平均
峰段电价	经验高价压力作用于窗口	检查任务移动与储能套利响应	不赋予价格概率
高碳	经验高碳强度压力	检查购电碳排目标是否改变动作	不代表政策情景预测
低新能源	压低可用新能源	检查正向购电和储能支撑需求	不外推到所有季节
联合压力	同时叠加低新能源与高碳等压力	检查多重约束下的非劣性	不称最坏情形上界

联合搜索按照当前基线评分选取候选任务，固定邻域之外的任务并生成一个可行移动。新排程确定后，重新计算六区域设施负荷，再分别求解各场景的储能追补子问题；是否接受候选，取决于硬约束、 ε 非劣性和归一化距离。任一场景出现任务遗漏、SLA、SOC、互斥或电能平衡失败时，候选立即被拒绝，不再比较软指标。

归一化指标差值按式 (27) 计算。

$$\Delta_m = \frac{F_m^{\text{cand}} - F_m^{\text{base}}}{\max\{|F_m^{\text{base}}|, s_m\}}, \quad m \in \{C, E, \bar{\ell}, P, -\rho_R\}, \quad (27)$$

其中 s_m 是预先确定的稳定尺度。候选需满足 $\Delta_m \leq \varepsilon_m$ ，且至少有一项指标严格改善；在此可接受集合内，再最小化 $\|\Delta\|_2$ 选择等权局部膝点。该处理先消除量纲差异，再限制极小尺度指标对结果的支配。

候选只有在满足硬约束、相对顺序基线的退化不超过允许范围且至少一项指标严格改善时才被接受，单项收益不等同于全面改善。

9.3 五场景结果

五场景结果见表 13。

表 13 Q4 五场景顺序基线与局部联合候选比较

场景	方案	净成本/CNY	购电碳/tCO ₂	新能源率	关键差值
观测	顺序基线	-13390080.274	0	0.686225	—
观测	联合候选	-13406141.443	0	0.686811	成本-16,061 CNY
峰段电价	顺序基线	-13390080.274	0	0.686225	—
峰段电价	联合候选	-13406141.443	0	0.686811	与观测场景一致
高碳	顺序基线	-13390080.274	0	0.686225	—
高碳	联合候选	-13406141.443	0	0.686811	与观测场景一致
低新能源	顺序基线	-12234343.344	10.947057	0.789435	—
低新能源	联合候选	-12275910.819	3.124382	0.790362	成本-41,567 CNY; 碳排-7.823 tCO ₂
联合压力	顺序基线	-12234340.410	12.511593	0.789435	—
联合压力	联合候选	-12275910.819	3.124382	0.790362	碳排-9.387 tCO ₂

局部联合候选满足任务、储能与电网约束。观测场景下，候选相对顺序基线的净运行成本差为-16,061 CNY。低新能源场景的成本差和购电碳排差分别为-41,567 CNY与-7.823 tCO₂；联合压力场景的购电碳排差为-9.387 tCO₂。负差值表示候选指标更低，成本仍按购电减售电收入计算。

图 11 在成本-碳排平面比较五个场景，点大小编码新能源利用率增益，点形和边框提供颜色之外的第二编码。碳排为零时不同场景可能重合，读图时需结合形状、边框和标注；精确数值以表 13 为准。

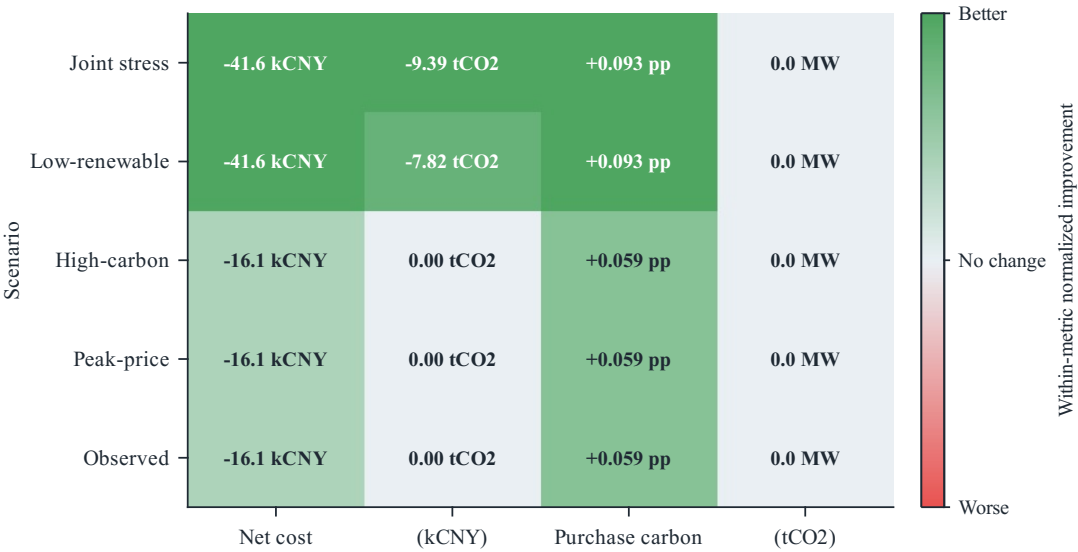


图 11 五场景下顺序基线与局部联合候选的成本-购电碳排权衡。点大小编码新能源利用率增益，点形和边框提供颜色之外的第二编码。

图 12 展示观测场景 72 h 窗口内，候选相对顺序基线的六区域聚合计算负荷与净购电差异。只绘制至少有一项差值非零的事件小时，并保留事件之间的真实时间间距；上下两面板使用独立纵轴，以免少数较大的净购电响应压缩任务侧调整。任务移动和储能追补共同决定结果，不能将净购电变化归因于单一任务移动。

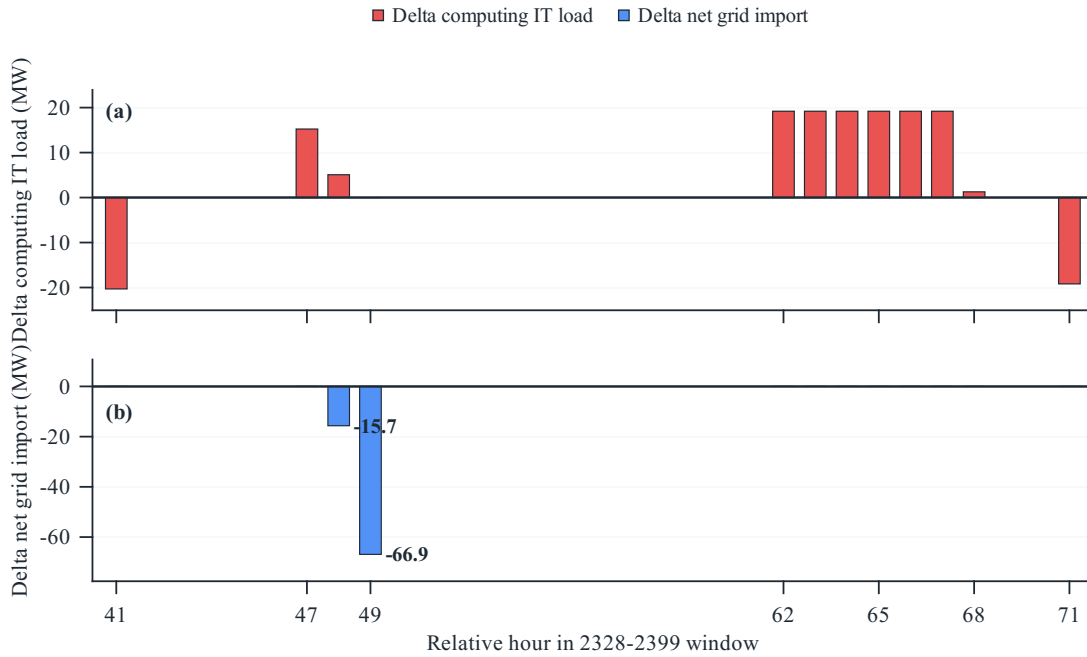


图 12 观测场景 72 h 窗口内局部联合候选相对顺序基线的事件响应。上图为六区域聚合计算负荷差，下图为六区域聚合净购电差；未绘制小时的两项差值均为零。净购电差为负表示候选减少净购电或增加净外送。

9.4 结果范围与最优性边界

在固定输入、种子和候选顺序下，Q4输出保持一致，确定性结果为未通过。五场景下任务、储能和电网约束均得到满足。由于搜索只覆盖 72 h 窗口和 12 条释放任务，且采用交替局部搜索，结果只解释为“72 小时、12 任务邻域的受限局部联合优化”，不代表 2407 h 全时域最优、全局 Pareto 最优或联合模型的唯一解。

10 综合评价、局限与推广

10.1 跨问题接口的一致性

四问共享任务唯一性、不可抢占、网络时延、GPU/功率和收尾边界。Q1 将原始任务汇总为需求统计，同时保留实际任务供排程使用；Q2 在相同任务约束上加入电价、碳强度和新能源；Q3 固定 Q2 负荷并加入储能状态与购售电；Q4 仅在声明的 72 h 窗口内重新开放少量任务。各问新增的变量和结论范围均可沿接口追踪，预测值、历史排程和能源口径不会被重复计入。

四问的结果均与同输出基线比较。Q1 给出盲测预测误差、区间覆盖和最后窗口可行排程；Q2 完成全时域任务调度并量化成本、碳排与时延的权衡；Q3 给出六区域滚动储能方案及其经济结果；Q4 在 72 h 代表性窗口内评估任务移动与储能响应的联动改善。各项结论均先满足任务、SLA、容量、SOC 和电能平衡约束，再解释软指标变化。

表 14 四问主要结果与适用范围

问题	求解方案	主要结果	适用范围
Q1	分层残差预测与 CP-SAT 排程	盲测窗口的系统 WAPE 为0.8934，经验覆盖率为0.9468；最后窗口任务完成率为1.0000。	预测用于容量压力判断，排程依据实际到达任务。
Q2	有界滚动交换	完成率为1.0000，SLA 违约率为0.0000；相对 FIFO 的成本、碳排和平均时延变化见表 9。	结论对应 2400 h 全时域和固定权重偏好。
Q3	六区域滚动二进制 MILP	覆盖 2407 h 时域；相对无储能基线，净成本变化为-38,899,970 CNY，购电侧碳排变化为-8.851 tCO ₂ 。	结论对应题面购售电边界，不代表全时域全局最优。
Q4	任务邻域与储能追补的局部联合优化	观测、低新能源和联合压力场景的成本或碳排差值见表 13；候选满足任务、储能和电网约束。	结论限定在 72 h 窗口和 12 条任务邻域。

10.2 模型优点

预测结果仅用于判断容量压力，最后窗口仍按实际到达任务排程，因此预测误差不会直接造成任务遗漏。18条底层序列经分层一致性校正后与区域总量、系统总量保持同一口径，共形区间则给出相应的不确定性范围。四问沿用同一套任务约束和功率复算接口，主模型与基线因此具有可比输出。能源侧的经济口径也保持明确：Q3 分别解释负成本、购电碳排和储能约束，Q4 只在固定窗口与任务邻域内解释任务-储能耦合，并保留明确的适用边界。

10.3 具体局限

Q1 的系统 WAPE 仍较高，合并校准区间也可能掩盖稀疏序列的覆盖不足。只有增加独立滚动起点，才能判断两阶段模型或分任务类型的直接多步模型是否稳定优于当前方法。CP-SAT 在固定时限内返回可行状态，当前最优性间隙尚不足以形成最优性证明；固定排程对需求和持续时间正向扰动的余量也有限。

Q2 采用固定权重和一次有界局部交换，未覆盖完整非支配前沿。静态网络时延没有计入拥塞、迁移流量、数据传输能耗和跨区费用；新能源利用率略降，表明当前权重更偏向成本与碳排。若应用侧更关注时延，应显式设置平均或分位数时延上限，并用 ε -constraint 生成候选方案。

Q3 的净售电收益取决于题面允许的购售电边界。模型没有追踪每单位售电来自新能源、储能还是同期购电，也没有计入电池退化、辅助服务、需量电费和终端 SOC 机会价值。若实际市场禁止购电转售，应增加能源来源追踪变量，或约束 g_{rt}^- 不超过同期新能源余量与可追踪储能放电；取得循环寿命参数后，再按吞吐量或等效循环计价引入退化成本。

Q4 只覆盖一个 72 h 窗口和 12 任务邻域，不能代表所有季节或拥塞状态。五类场景是确定性压力测试，不是概率情景，也没有给出风险分布；顺序基线与联合候选还继承了 Q2、Q3 的市场和网络简化。当前结果仅支持该窗口与邻域内的局部联动改善，不能外推为全时域联合调度的工程保证。

10.4 推广与执行建议

实际部署可采用分层滚动架构：日级更新预测区间，小时级生成柔性任务候选，分钟级修正局部排程；高价、高碳、峰值或新能源短缺时启动储能联合搜索。补充数据和市场规则后，可进一步引入时延约束、储能退化与能源溯源，并扩展联合任务邻域。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于思路讨论、代码调试、图表制作、文字润色和结果复核等辅助工作，详细使用情况见附录。

参考文献

- [1] 钱声攀, 邱奔, 李哲, 王少鹏. 数据中心能效优化策略研究 [J]. 信息通信技术与政策, 2021, 47(4): 19–26. DOI: <https://doi.org/10.12267/j.issn.2096-5931.2021.04.004>.
- [2] 李桃. 绿色数据中心环境下大数据处理的能耗优化策略研究 [J]. 电脑知识与技术, 2026, 22(13): 56–58. DOI: <https://doi.org/10.14004/j.cnki.ckt.2026.0622>.
- [3] Bergmeir C, Hyndman R J, Koo B. A note on the validity of cross-validation for evaluating autoregressive time series prediction[J]. *Computational Statistics & Data Analysis*, 2018, 120: 70–83. DOI: <https://doi.org/10.1016/j.csda.2017.11.003>.
- [4] Hyndman R J, Athanasopoulos G. *Forecasting: Principles and Practice*[M/OL]. 3rd ed. Melbourne: OTexts, 2021. <https://otexts.com/fpp3/>.
- [5] Friedman J H. Greedy function approximation: A gradient boosting machine[J]. *The Annals of Statistics*, 2001, 29(5): 1189–1232. DOI: <https://doi.org/10.1214/aos/1013203451>.
- [6] Hastie T, Tibshirani R, Friedman J. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*[M]. 2nd ed. New York: Springer, 2009.
- [7] 梁毅, 曾绍康, 梁岩德, 丁毅. 一种基于周期性特征的数据中心在线负载资源预测方法 [J]. 计算机工程与科学, 2020, 42(3): 381–390.
- [8] Wickramasuriya S L, Athanasopoulos G, Hyndman R J. Optimal forecast reconciliation for hierarchical and grouped time series through trace minimization[J]. *Journal of the American Statistical Association*, 2019, 114(526): 804–819. DOI: <https://doi.org/10.1080/01621459.2018.1448825>.
- [9] Vovk V, Gammerman A, Shafer G. *Algorithmic Learning in a Random World*[M]. New York: Springer, 2005.
- [10] Romano Y, Patterson E, Candès E J. Conformalized quantile regression[C]// *Advances in Neural Information Processing Systems*. 2019, 32: 3543–3553.

- [11] Perron L, Furnon V. OR-Tools: Google optimization tools[EB/OL]. Google Developers, 2026[2026-08-09]. <https://developers.google.com/optimization/>.
- [12] 钱敏. 云计算背景下数据中心资源调度方法研究 [J]. 信息技术, 2022(5): 188–192. DOI: <https://doi.org/10.13274/j.cnki.hdzj.2022.05.032>.
- [13] 王立红, 张延华, 孟德彬, 李萌. 基于 DDPG 算法的云数据中心任务节能调度研究 [J]. 高技术通讯, 2023, 33(9): 927–936. DOI: <https://doi.org/10.3772/j.issn.1002-0470.2023.09.004>.
- [14] 刘陈伟, 孙鉴, 雷冰冰, 徐涛, 吴佳伟. 基于改进粒子群算法的云数据中心能耗优化任务调度策略 [J]. 计算机科学, 2023, 50(7): 246–253. DOI: <https://doi.org/10.11896/jsj.kx.220900176>.
- [15] 孙湛冬, 焦娇, 李伟, 李志鹏, 李鹏恩. 基于改进蚁群算法的电力云数据中心任务调度策略研究 [J]. 电力系统保护与控制, 2022, 50(2): 95–101. DOI: <https://doi.org/10.19783/j.cnki.pspc.210466>.
- [16] Wolsey L A. *Integer Programming*[M]. New York: Wiley, 1998.
- [17] Pinedo M L. *Scheduling: Theory, Algorithms, and Systems*[M]. 5th ed. Cham: Springer, 2016.
- [18] 王飞, 徐健, 李伟, 汪新浩, 施啸寒. 基于分布式储能系统的风储滚动优化调度方法 [J]. 山东大学学报 (工学版), 2017, 47(6): 89–94. DOI: <https://doi.org/10.6040/j.issn.1672-3961.0.2017.208>.
- [19] Sioshansi R, Denholm P, Jenkin T, Weiss J. Estimating the value of electricity storage in PJM: Arbitrage and some welfare effects[J]. *Energy Economics*, 2009, 31(2): 269–277. DOI: <https://doi.org/10.1016/j.eneco.2008.10.005>.
- [20] OpenAI Codex, GPT-5, OpenAI, 2026-08-07 至 2026-08-09。

A 环境与复现命令

附件源程序以 Python 为模型与审计入口，MATLAB R2026a 负责正式图件及图件数据准备。程序只读取官方附件，不引入第三方数据；默认随机种子为 20260801。

表 15 附件复现环境与固定接口

组件	固定版本或接口	用途
Python	3.13.x; 依赖见 <code>requirements.txt</code>	四问模型、同输出基线和审计
数值求解	OR-Tools CP-SAT; SciPy/HiGHS MILP	Q1 排程、Q3 储能与 Q4 追补
数据接口	pandas、NumPy、openpyxl	表格读取、字段转换和确定性派生
绘图	MATLAB R2026a	图件数据准备与 12 张正式图三格式导出
输入与输出	官方附件目录、 <code>resul</code> ts/、 <code>manifest/</code>	数据校验、结果摘要和来源定位

模型运行使用官方附件目录作为输入；验证模式只检查依赖、输入文件名和 SHA-256，不启动长时间求解。

```
python -m pip install -r requirements.txt
python run.py --question Q1 --input-dir <official-input-dir> --output-dir <output-dir> --seed 20260801
python run.py --question Q2 --input-dir <official-input-dir> --output-dir <output-dir> --seed 20260801
python run.py --question Q3 --input-dir <official-input-dir> --output-dir <output-dir> --seed 20260801
python run.py --question Q4 --input-dir <official-input-dir> --output-dir <output-dir> --seed 20260801
python run.py --question Q1 --input-dir <official-input-dir> --verify-only
python run.py --question Q2 --input-dir <official-input-dir> --verify-only
python run.py --question Q3 --input-dir <official-input-dir> --verify-only
python run.py --question Q4 --input-dir <official-input-dir> --verify-only
matlab -batch "cd('code/matlab'); build_all_figures"
Get-Content manifest/package_manifest.sha256
```

Listing 1 四问主入口与输入复核命令

B 六组关键算法代码

以下代码均从附件中的正式源文件按命名标记截取，附录不复制或改写实现。标记行不计入展示内容，行号对应源文件原始行号；代码框可跨页断开，便于审阅完整函数体。

B.1 Q1 需求预测：层级一致性与区间校准

```

223 def reconcile_forecast(
224     history: np.ndarray, baseline: np.ndarray, residual: np.ndarray, lag: int, shrinkage: float,
225     ↪ origin: int
226 ) -> np.ndarray:
227     variance = reconciliation_weights(history, lag)
228     weight = np.sqrt(variance)
229     matrix = S_MATRIX / weight[:, None]
230     aggregate_baseline = node_values(seasonal_prediction(history, origin, len(baseline), lag))[:, len(
231     ↪ SERIES):]
232     reconciled: list[np.ndarray] = []
233     for step in range(len(baseline)):
234         bottom_base = np.maximum(0.0, baseline[step] + shrinkage * residual[step])
235         all_base = np.concatenate([bottom_base, aggregate_baseline[step]])
236         result = lsq_linear(matrix, all_base / weight, bounds=(0.0, np.inf), method="trf", lsqr_tol="
237         ↪ auto")
238         if not result.success or not np.isfinite(result.x).all():
239             raise RuntimeError(f"hierarchical reconciliation failed: {result.message}")
240         reconciled.append(result.x)
241     return np.vstack(reconciled)

```

Listing 2 层级预测的一致性校正函数

```

301 def conformal_calibration(actual: np.ndarray, prediction: np.ndarray) -> tuple[np.ndarray, float]:
302     absolute_error = np.abs(actual - prediction)
303     scale = np.maximum.reduce([
304         np.median(absolute_error, axis=0),
305         np.mean(actual, axis=0) * 0.05,
306         np.full(actual.shape[1], 1e-6),
307     ])
308     scores = (absolute_error / scale).ravel()
309     n = len(scores)
310     level = min(1.0, np.ceil((n + 1) * (1 - ALPHA)) / n)
311     quantile = float(np.quantile(scores, level, method="higher"))
312     return scale * quantile, quantile

```

Listing 3 盲测残差的 pooled conformal 校准函数

B.2 Q1 实际任务排程：CP-SAT 可选区间与容量约束

```

287 def cp_sat_schedule(tasks: pd.DataFrame, context: dict, carry: pd.DataFrame, max_seconds: float =
288     ↪ 90.0) -> tuple[pd.DataFrame, dict]:
289     model = cp_model.CpModel()
290     resources = {region: {"gpu": [], "it": [], "facility": []} for region in context["regions"]}
291     for region in context["regions"]:
292         index = context["index"][region]
293         for hour in range(Q1_START_H, HORIZON_END_H):
294             start, end = hour * 60, (hour + 1) * 60
295             background = int(round(context["nonai"][index, start] * 1000))
296             interval = model.NewIntervalVar(start, 60, end, f"background_{region}_{hour}")
297             resources[region]["it"].append((interval, background))
298             resources[region]["facility"].append((interval, int(round(background * context["pue"] [
299     ↪ region]))))
300     for _, task in carry.iterrows():
301         region = str(task.ExecutionRegion)
302         start, end = max(FINAL_START_M, int(task.StartMinute)), min(FINAL_END_M, int(task.EndMinute))
303         if end <= start:
304             continue
305         duration = end - start
306         interval = model.NewIntervalVar(start, duration, end, f"carry_{int(task.TaskID)}")
307         demand = int(task.GPU_Demand)
308         power = context["power"][str(task.TaskType)]
309         resources[region]["gpu"].append((interval, demand))
310         resources[region]["it"].append((interval, int(round(demand * power * 1000))))
311         resources[region]["facility"].append((interval, int(round(demand * power * context["pue"] [
312     ↪ region] * 1000))))
313     choices: dict[int, list[tuple]] = {}

```

```

314 for _, task in tasks.iterrows():
315     task_id = int(task.TaskID)
316     duration = int(task.EstimatedDuration_min)
317     arrival = max(FINAL_START_M, int(task.ArrivalHour) * 60, int(task.EarliestStartHour) * 60)
318     latest = min(FINAL_END_M, int(task.LatestFinishHour) * 60)
319     task_type = str(task.TaskType)
320     power = context["power"][task_type]
321     task_choices: list[tuple] = []
322     for region in candidate_regions(task, context):
323         if latest - duration < arrival:
324             continue
325         start_upper = arrival if task_type == "RealTimeInference" else latest - duration
326         start = model.NewIntVar(arrival, start_upper, f"start_{task_id}_{region}")
327         end = model.NewIntVar(arrival + duration, latest, f"end_{task_id}_{region}")
328         use = model.NewBoolVar(f"use_{task_id}_{region}")
329         model.Add(end == start + duration)
330         interval = model.NewOptionalIntervalVar(start, duration, end, use, f"job_{task_id}_{region}
↪ )")
331         demand = int(task.GPU_Demand)
332         resources[region]["gpu"].append((interval, demand))
333         resources[region]["it"].append((interval, int(round(demand * power * 1000))))
334         resources[region]["facility"].append((interval, int(round(demand * power * context["pue"]
↪ [region] * 1000))))
335         task_choices.append((use, start, end, region))
336         if not task_choices:
337             return pd.DataFrame(), {"status": "INFEASIBLE_NO_CANDIDATE", "task_count": int(len(tasks))
↪ , "max_time_seconds": max_seconds}
338     model.AddExactlyOne([choice[0] for choice in task_choices])
339     choices[task_id] = task_choices

```

Listing 4 Q1 CP-SAT 排程、GPU/IT/设施功率和收尾边界

B.3 Q2 全时域调度：有界滚动交换

```

565
566 for block_start_hour in range(0, MAIN_END_HOUR, CHUNK_HOURS):
567     block_end_hour = min(MAIN_END_HOUR, block_start_hour + CHUNK_HOURS)
568     chunk = tasks[
569         (~tasks.TaskType.eq("RealTimeInference"))
570         & tasks.ArrivalHour.ge(block_start_hour)
571         & tasks.ArrivalHour.lt(block_end_hour)
572     ].copy()
573     if chunk.empty:
574         continue
575     chunk["_work"] = chunk.GPU_Demand * chunk._duration
576     chunk["_slack"] = chunk._latest_start - chunk._earliest
577     chunk["_priority"] = chunk.TaskType.map(task_priority)
578     chunk = chunk.sort_values(
579         ["_work", "_slack", "_priority", "TaskID"],
580         ascending=[False, False, True, True],
581     )
582     for _, task in chunk.iterrows():
583         task_id = int(task.TaskID)
584         old = rows[task_id]
585         ledger.apply(old, -1)
586         old_metrics = option_metrics(
587             task,
588             str(old["ExecutionRegion"]),
589             int(old["StartMinute"]),
590             ledger,
591             context,
592         )
593         latency_ceiling = min(
594             float(task.MaxLatency_ms),
595             float(old["NetworkLatency_ms"]) + POLICY["latency_epsilon_ms"],
596         )
597         options: list[tuple[float, str, int, dict[str, float]]] = []
598         for region in eligible_regions(task, context):
599             latency = context["latency"][(str(task.SourceRegion), region)]
600             if latency > latency_ceiling + TOL:

```

```

601         continue
602     for begin in candidate_starts(task, region, old, context):
603         evaluated_options += 1
604         metrics = option_metrics(task, region, begin, ledger, context)
605         options.append((metrics["score"], region, begin, metrics))
606     options.sort(key=lambda item: (item[0], item[3]["latency_ms"], item[2], item[1]))

```

Listing 5 候选任务的区域-时刻枚举、增量审计与确定性接受规则

B.4 Q3 储能优化：带互斥变量的滚动 MILP

```

171
172     eq_rows = 4 * horizon
173     ub_rows = 3 * horizon + 1
174     matrix = lil_matrix((eq_rows + ub_rows, nvar), dtype=float)
175     row_lower = np.empty(eq_rows + ub_rows)
176     row_upper = np.empty(eq_rows + ub_rows)
177     row = 0
178     for t in range(horizon):
179         matrix[row, slices["soc"].start + t] = 1.0
180         matrix[row, slices["charge"].start + t] = -eta_c
181         matrix[row, slices["discharge"].start + t] = 1.0 / eta_d
182         if t > 0:
183             matrix[row, slices["soc"].start + t - 1] = -1.0
184             rhs = 0.0
185         else:
186             rhs = initial_soc
187         row_lower[row] = rhs
188         row_upper[row] = rhs
189         row += 1
190
191         matrix[row, slices["charge"].start + t] = 1.0
192         matrix[row, slices["renewable_charge"].start + t] = -1.0
193         matrix[row, slices["grid_charge"].start + t] = -1.0
194         row_lower[row] = row_upper[row] = 0.0
195         row += 1
196
197         matrix[row, slices["renewable_to_load"].start + t] = 1.0
198         matrix[row, slices["renewable_charge"].start + t] = 1.0
199         matrix[row, slices["sell"].start + t] = 1.0
200         matrix[row, slices["curtail"].start + t] = 1.0
201         row_lower[row] = row_upper[row] = renewable[t]
202         row += 1
203
204         matrix[row, slices["renewable_to_load"].start + t] = 1.0
205         matrix[row, slices["discharge"].start + t] = 1.0
206         matrix[row, slices["grid_load"].start + t] = 1.0
207         row_lower[row] = row_upper[row] = float(data.RecomputedTotalLoad_MW.iloc[t])
208         row += 1

```

Listing 6 储能状态转移、电能平衡、购售电和终端状态约束

B.5 Q3 完整审计：充放电与购售电双互斥复核

```

169
170     extended_constraints = LinearConstraint(
171         vstack([original_matrix, mutex_matrix.tocsr()], format="csr"),
172         np.concatenate([np.asarray(constraints.lb), mutex_lower]),
173         np.concatenate([np.asarray(constraints.ub), mutex_upper]),
174     )
175
176     started = time.perf_counter()
177     result = milp(
178         c=extended_objective,
179         integrality=extended_integrality,
180         bounds=extended_bounds,
181         constraints=extended_constraints,
182         options={"presolve": True, "time_limit": TIME_LIMIT_S, "mip_rel_gap": 1e-7},
183     )
184     runtime = time.perf_counter() - started
185     info = {
186         "status": str(result.message),
187         "success": bool(result.success and result.x is not None),
188         "runtime_s": runtime,
189         "mip_gap": q3.finite(getattr(result, "mip_gap", None)),
190         "objective": q3.finite(getattr(result, "fun", None)),
191         "mip_node_count": int(getattr(result, "mip_node_count", 0) or 0),
192         "solver_mode": "full_cycle_binary_MILP_double_mutex_audit_probe",
193     }
194     if not info["success"]:
195         return pd.DataFrame(), info

```

Listing 7 六区域全时域二进制 MILP 可行性探针

B.6 Q4 受限联合优化：任务邻域与储能追补

```

122 def optimize(q2, q4, tasks, context, incumbent, energy_inputs):
123     region_time, storage, pue, power_map = energy_inputs
124     rows = {int(row["TaskID"]): dict(row) for row in incumbent.to_dict("records")}
125     ledger = q2.MinuteLedger(context)
126     for row in rows.values(): ledger.apply(row, +1)
127     current, envelope_meta = evaluate(q4, incumbent, region_time, storage, pue, power_map)
128     scales = {name: {"cost": max(abs(z["metrics"]["cost_CNY"]), 1), "carbon": max(abs(z["metrics"]["
    ↪ carbon_tCO2"]), 1), "peak": max(abs(z["metrics"]["positive_peak_MW"]), 1)} for name, z in
    ↪ current.items()}
129     current_objective, trace, evaluated = objective(current, scales), [], 0
130     task_map = tasks.set_index("TaskID", drop=False)
131     for task_id in select_queue(incumbent, tasks, context).TaskID.astype(int):
132         task, old = task_map.loc[task_id], rows[task_id]
133         ledger.apply(old, -1)
134         options = []
135         latency_ceiling = min(float(task.MaxLatency_ms), float(old["NetworkLatency_ms"]) +
    ↪ LATENCY_GUARD_MS)
136         for region in q2.eligible_regions(task, context):
137             latency = context["latency"][(str(task.SourceRegion), region)]
138             if latency > latency_ceiling + q2.TOL: continue
139             starts = {int(old["StartMinute"]), int(task.earliest), *(int(old["StartMinute"]) + delta
    ↪ for delta in (-120, -60, 60, 120))}
140             for begin in sorted(starts):
141                 if begin < int(task.earliest) or begin > int(task._latest_start) or begin + int(task.
    ↪ _duration) > H1 * 60: continue
142                 if not ledger.can_place(task, region, begin): continue
143                 proxy = q2.option_metrics(task, region, begin, ledger, context)
144                 options.append((proxy["score"], proxy["latency_ms"], begin, region))
145             options = sorted(set(options), key=lambda x: (x[0], x[1], x[2], x[3]))[:SHORTLIST_SIZE]
146             if (str(old["ExecutionRegion"]), int(old["StartMinute"])) not in {(x[3], x[2]) for x in
    ↪ options}:
147                 options.append((float("inf"), float(old["NetworkLatency_ms"]), int(old["StartMinute"]),
    ↪ str(old["ExecutionRegion"])))
148             trials = []
149             for _proxy, _latency, begin, region in options:
150                 candidate_row = dict(old)

```

```

151     candidate_row.update({"ExecutionRegion": region, "StartMinute": begin, "EndMinute": begin
↪ + int(task._duration), "NetworkLatency_ms": context["latency"][(str(task.SourceRegion), region)
↪ ], "Method": METHOD})
152     rows[task_id] = candidate_row
153     schedule = pd.DataFrame(rows.values(), columns=q2.SCHEDULE_COLUMNS).sort_values("TaskID").
↪ reset_index(drop=True)
154     result, _meta = evaluate(q4, schedule, region_time, storage, pue, power_map)
155     evaluated += 1
156     trials.append((objective(result, scales), region, begin, candidate_row, result))
157     best = min(trials, key=lambda x: (x[0], x[1], x[2]))
158     accepted = best[0] < current_objective - 1e-10
159     if accepted:
160         rows[task_id], current, current_objective = best[3], best[4], best[0]
161         ledger.apply(best[3], +1)
162     else:
163         rows[task_id] = old
164         ledger.apply(old, +1)
165     trace.append({"TaskID": task_id, "options_evaluated": len(trials), "accepted": accepted, "
↪ old_region": old["ExecutionRegion"], "new_region": rows[task_id]["ExecutionRegion"], "
↪ old_start_minute": int(old["StartMinute"]), "new_start_minute": int(rows[task_id]["StartMinute"]
↪ ), "joint_objective": current_objective})
166     schedule = pd.DataFrame(rows.values(), columns=q2.SCHEDULE_COLUMNS).sort_values("TaskID").
↪ reset_index(drop=True)
167     return schedule, current, pd.DataFrame(trace), envelope_meta, evaluated

```

Listing 8 固定-优化任务邻域和储能响应评价

```

205 def solve_integrated(
206     frame: pd.DataFrame,
207     storage_table: pd.DataFrame,
208 ) -> tuple[pd.DataFrame, dict[str, Any]]:
209     data = frame.sort_values(["Region", "Hour"]).reset_index(drop=True)
210     regions = sorted(data.Region.astype(str).unique())
211     stores = storage_map(storage_table)
212     region_frames = {
213         region: data[data.Region.astype(str) == region].sort_values("Hour").reset_index(drop=True)
214         for region in regions
215     }
216     if any(len(region_frames[region]) != HORIZON for region in regions):
217         raise ValueError("candidate envelope must contain 72 rows for every region")
218
219     region_count = len(regions)
220     n_rt = region_count * HORIZON
221     variable_names = [
222         "charge",
223         "discharge",
224         "soc",
225         "renewable_to_load",
226         "renewable_charge",
227         "grid_charge",
228         "grid_load",
229         "sell",
230         "curtail",
231         "charge_mode",
232         "grid_mode",
233     ]
234     slices: dict[str, slice] = {}
235     cursor = 0
236     for name in variable_names:
237         slices[name] = slice(cursor, cursor + n_rt)
238         cursor += n_rt
239     system_peak = cursor
240     nvar = cursor + 1
241
242     def at(name: str, region_index: int, hour: int) -> int:
243         return slices[name].start + region_index * HORIZON + hour
244
245     objective = np.zeros(nvar)
246     lower = np.zeros(nvar)
247     upper = np.full(nvar, np.inf)
248     integrality = np.zeros(nvar)
249     total_nominal_cost = 0.0

```

```

250 total_nominal_carbon = 0.0
251 total_renewable = 0.0
252 total_import_cap = 0.0
253 for region in regions:
254     d = region_frames[region]
255     total_nominal_cost += float(np.dot(d.Total_Load_MW, d.ElectricityPrice_CNY_per_MWh))
256     total_nominal_carbon += float(np.dot(d.Total_Load_MW, d.CarbonIntensity_tCO2_per_MWh))
257     total_renewable += float(d.AvailableRenewable_MW.sum())
258     total_import_cap += float(stores[region].MaxGridImport_MW)
259 cost_scale = max(total_nominal_cost, 1.0)
260 carbon_scale = max(total_nominal_carbon, 1.0)
261 renewable_scale = max(total_renewable, 1.0)
262
263 for ri, region in enumerate(regions):
264     d = region_frames[region]
265     st = stores[region]
266     max_charge = float(st.MaxChargePower_MW)
267     max_discharge = float(st.MaxDischargePower_MW)
268     import_cap = float(st.MaxGridImport_MW)
269     export_cap = min(float(st.SellLimit_MW), float(st.MaxGridExport_MW))
270     for hour in range(HORIZON):
271         charge = at("charge", ri, hour)
272         discharge = at("discharge", ri, hour)
273         objective[at("grid_load", ri, hour)] = (
274             float(d.ElectricityPrice_CNY_per_MWh.iloc[hour]) / cost_scale
275             + CARBON_WEIGHT
276             * float(d.CarbonIntensity_tCO2_per_MWh.iloc[hour])
277             / carbon_scale
278         )
279         objective[at("grid_charge", ri, hour)] = objective[at("grid_load", ri, hour)]
280         objective[at("sell", ri, hour)] = -float(d.SellPrice_CNY_per_MWh.iloc[hour]) / cost_scale
281         objective[at("curtail", ri, hour)] = 0.03 / renewable_scale
282         objective[charge] += 1e-7
283         objective[discharge] += 1e-7
284         upper[charge] = max_charge
285         upper[discharge] = max_discharge
286         lower[at("soc", ri, hour)] = float(st.MinSOC_MWh)
287         upper[at("soc", ri, hour)] = float(st.StorageCapacity_MWh)
288         upper[at("renewable_charge", ri, hour)] = max_charge
289         upper[at("grid_charge", ri, hour)] = max_charge
290         upper[at("grid_load", ri, hour)] = import_cap
291         upper[at("sell", ri, hour)] = export_cap
292         upper[at("charge_mode", ri, hour)] = 1.0
293         upper[at("grid_mode", ri, hour)] = 1.0
294         integrality[at("charge_mode", ri, hour)] = 1
295         integrality[at("grid_mode", ri, hour)] = 1
296     objective[system_peak] = PEAK_WEIGHT / max(total_import_cap, 1.0)
297     lower[system_peak] = 0.0
298     upper[system_peak] = total_import_cap
299
300 constraint_count = 8 * n_rt + region_count + HORIZON
301 matrix = lil_matrix((constraint_count, nvar))
302 lo = np.full(constraint_count, -np.inf)
303 hi = np.full(constraint_count, np.inf)
304 row = 0
305 for ri, region in enumerate(regions):
306     d = region_frames[region]
307     st = stores[region]
308     initial = float(st.InitialSOC_MWh)
309     eta_charge = float(st.ChargeEfficiency)
310     eta_discharge = float(st.DischargeEfficiency)
311     max_charge = float(st.MaxChargePower_MW)
312     max_discharge = float(st.MaxDischargePower_MW)
313     import_cap = float(st.MaxGridImport_MW)
314     export_cap = min(float(st.SellLimit_MW), float(st.MaxGridExport_MW))
315     for hour in range(HORIZON):
316         matrix[row, at("soc", ri, hour)] = 1
317         matrix[row, at("charge", ri, hour)] = -eta_charge
318         matrix[row, at("discharge", ri, hour)] = 1 / eta_discharge
319         if hour:
320             matrix[row, at("soc", ri, hour - 1)] = -1
321         lo[row] = hi[row] = 0.0

```



```

322         else:
323             lo[row] = hi[row] = initial
324             row += 1
325
326             matrix[row, at("charge", ri, hour)] = 1
327             matrix[row, at("renewable_charge", ri, hour)] = -1
328             matrix[row, at("grid_charge", ri, hour)] = -1
329             lo[row] = hi[row] = 0.0
330             row += 1
331
332             matrix[row, at("renewable_to_load", ri, hour)] = 1
333             matrix[row, at("renewable_charge", ri, hour)] = 1
334             matrix[row, at("sell", ri, hour)] = 1
335             matrix[row, at("curtail", ri, hour)] = 1
336             lo[row] = hi[row] = float(d.AvailableRenewable_MW.iloc[hour])
337             row += 1
338
339             matrix[row, at("renewable_to_load", ri, hour)] = 1
340             matrix[row, at("discharge", ri, hour)] = 1
341             matrix[row, at("grid_load", ri, hour)] = 1
342             lo[row] = hi[row] = float(d.Total_Load_MW.iloc[hour])
343             row += 1
344
345             matrix[row, at("charge", ri, hour)] = 1
346             matrix[row, at("charge_mode", ri, hour)] = -max_charge
347             hi[row] = 0.0
348             row += 1
349
350             matrix[row, at("discharge", ri, hour)] = 1
351             matrix[row, at("charge_mode", ri, hour)] = max_discharge
352             hi[row] = max_discharge
353             row += 1
354
355             matrix[row, at("grid_load", ri, hour)] = 1
356             matrix[row, at("grid_charge", ri, hour)] = 1
357             matrix[row, at("grid_mode", ri, hour)] = -import_cap
358             hi[row] = 0.0
359             row += 1
360
361             matrix[row, at("sell", ri, hour)] = 1
362             matrix[row, at("grid_mode", ri, hour)] = export_cap
363             hi[row] = export_cap
364             row += 1
365
366             matrix[row, at("soc", ri, HORIZON - 1)] = 1
367             lo[row] = hi[row] = initial
368             row += 1
369
370     for hour in range(HORIZON):
371         for ri in range(region_count):
372             matrix[row, at("grid_load", ri, hour)] = 1
373             matrix[row, at("grid_charge", ri, hour)] = 1
374             matrix[row, at("sell", ri, hour)] = -1
375             matrix[row, system_peak] = -1
376             hi[row] = 0.0
377             row += 1
378     if row != constraint_count:
379         raise RuntimeError(f"constraint assembly mismatch: expected={constraint_count}, actual={row}")
380
381     started = time.perf_counter()
382     result = milp(
383         c=objective,
384         integrality=integrality,
385         bounds=Bounds(lower, upper),
386         constraints=LinearConstraint(matrix.tocsr(), lo, hi),
387         options={"presolve": True, "time_limit": 60.0, "mip_rel_gap": 1e-7},
388     )
389     runtime_s = time.perf_counter() - started
390     solver = {
391         "success": bool(result.success and result.x is not None),
392         "status": str(result.message),
393         "runtime_s": runtime_s,

```

```

394     "mip_gap": None if result.x is None else float(getattr(result, "mip_gap", 0.0) or 0.0),
395     "mip_node_count": None
396     if result.x is None
397     else int(getattr(result, "mip_node_count", 0) or 0),
398     "objective": None if result.fun is None else float(result.fun),
399     "solver_mode": "integrated_binary_MILP",
400 }
401 if not solver["success"]:
402     return pd.DataFrame(), solver
403
404 x = result.x
405 pieces: list[pd.DataFrame] = []
406 for ri, region in enumerate(regions):
407     out = region_frames[region].copy()
408     out["ChargePower_MW"] = x[slices["charge"]][ri * HORIZON : (ri + 1) * HORIZON]
409     out["DischargePower_MW"] = x[slices["discharge"]][ri * HORIZON : (ri + 1) * HORIZON]
410     out["SOC_MWh"] = x[slices["soc"]][ri * HORIZON : (ri + 1) * HORIZON]
411     out["RenewableToLoad_MW"] = x[slices["renewable_to_load"]][
412         ri * HORIZON : (ri + 1) * HORIZON
413     ]
414     out["RenewableCharge_MW"] = x[slices["renewable_charge"]][
415         ri * HORIZON : (ri + 1) * HORIZON
416     ]
417     out["GridCharge_MW"] = x[slices["grid_charge"]][ri * HORIZON : (ri + 1) * HORIZON]
418     out["GridLoadPurchase_MW"] = x[slices["grid_load"]][
419         ri * HORIZON : (ri + 1) * HORIZON
420     ]
421     out["GridPurchase_MW"] = out.GridLoadPurchase_MW + out.GridCharge_MW
422     out["GridSell_MW"] = x[slices["sell"]][ri * HORIZON : (ri + 1) * HORIZON]
423     out["Curtailement_MW"] = x[slices["curtail"]][ri * HORIZON : (ri + 1) * HORIZON]
424     out["NetGridImport_MW"] = out.GridPurchase_MW - out.GridSell_MW
425     out["ChargeMode"] = np rint(
426         x[slices["charge_mode"]][ri * HORIZON : (ri + 1) * HORIZON]
427     ).astype(int)
428     out["GridImportMode"] = np rint(
429         x[slices["grid_mode"]][ri * HORIZON : (ri + 1) * HORIZON]
430     ).astype(int)
431     out["Method"] = "Q2_candidate_plus_integrated_storage_MILP"
432     pieces.append(out)
433 dispatch = pd.concat(pieces, ignore_index=True).sort_values(["Region", "Hour"]).reset_index(drop=
    ↪ True)
434 solver["system_peak_variable_MW"] = float(x[system_peak])
435 solver["system_peak_net_import_MW"] = float(
436     dispatch.groupby("Hour").NetGridImport_MW.sum().max()
437 )
438 return dispatch, solver

```

Listing 9 Q4 场景下的储能 recourse 求解

C 源程序索引

源程序索引以提交包内的相对路径为准。manifest/source_map.json 记录入口、问题归属、作用和源文件 SHA-256；manifest/package_manifest.sha256 对附件文件逐项校验。run.py 是四问统一入口，核心实现按问题分目录保存。

表 16 Python 模型与审计源程序索引

相对路径	问题/角色	说明
run.py	Q1–Q4 统一入口	参数解析、种子和子程序分发
code/common/runtime.py	公共运行时	输入校验、路径和校验值工具
code/q1/forecast_core.py	Q1 预测	季节基线、HGBR 残差、层级校正和区间

相对路径	问题/角色	说明
code/q1/schedule_core.py	Q1 排程	暖机、CP-SAT、同约束基线和审计
code/q2/scheduler_core.py	Q2 调度	FIFO、滚动交换、逐时指标和复跑比较
code/q3/storage_core.py	Q3 储能	滚动 MILP、无储能基线和逐时审计
code/q3/full_audit_core.py	Q3 完整审计	区域全时域二进制互斥探针和复跑
code/q4/joint_core.py	Q4 联合主问题	任务 fix-and-optimize 与五情景评价
code/q4/storage_recourse.py	Q4 追补子问题	六区域储能响应、成本和碳排指标

表 17 MATLAB 图件源程序索引

相对路径或目录	类型	输出
code/matlab/prepare_figure_data.m	数据准备	12 张图的派生数据和输入哈希
code/matlab/build_all_figures.m	总构建入口	批量调用图件脚本和三格式导出
code/matlab/figures/	12 个正式图脚本	Q1–Q4 PDF、SVG、400 dpi PNG
code/matlab/helpers/	公共样式与导出 helper	尺寸、配色、图例、裁切和校验

D 结果与审计索引

结果目录只保存精简摘要和硬约束审计；长排程、逐时剖面、滚动块明细和场景逐时表均由程序按需生成，不装入提交包。

表 18 四问结果与审计文件索引

目录	核心文件或字段	复核内容
results/q1/	summary.json、constraint_audit.json、optimized_schedule.csv	盲测指标、538 条实际任务、资源和收尾边界
results/q2/	summary.json、constraint_audit.json	50000/50000 任务、SLA、成本、碳排、时延和复跑
results/q3/	summary.json、constraint_audit.json、full_audit_summary.json	SOC、互斥、电能平衡、90 块覆盖和 270/270 审计
results/q4/	summary.json、constraint_audit.json、scenario_metrics.csv	72 h、12 任务邻域、储能追补和确定性复跑

目录	核心文件或字段	复核内容
manifest/	source_map.json、package_manifest.sha256	源程序映射、文件哈希和发布白名单

E 附件复现与发布边界

附件只包含匿名化复现所需的正式源程序、依赖锁定、输入说明、精简结果摘要、审计摘要、图件脚本和文件哈希；原始数据、长排程和逐时结果不进入提交包。账号、密钥、个人或学校信息、临时日志、编辑器缓存、未采用候选和与主张无关的中间文件不进入提交包。

正式发布前逐项核对：源码路径可从 manifest/source_map.json 反查；每个输出均可由运行清单定位到输入、种子、环境和命令；package_manifest.sha256 与最终压缩包一致；论文中的图表和主张均能回溯到冻结结果。若输入、源码、结果或哈希发生漂移，应停止封存并重新执行相应问题的证据复核。

F 人工智能工具使用详情

F.1 所用 AI 工具名称和版本

表 19 AI 工具信息

序号	AI 工具名称	版本/型号	开发机构/公司	使用日期
1	OpenAI Codex	GPT-5	OpenAI	2026-08-07 至 2026-08-10

F.2 具体使用目的和环节

表 20 AI 工具的使用环节与辅助目的

序号	使用环节	使用目的	使用工具
1	候选方案与实验审计	梳理备选方案、检查基线可比性、协助定位约束审计和复跑中的实现问题	OpenAI Codex
2	程序调试	辅助检查 Python、OR-Tools、HiGHS 与 MATLAB 脚本的语法、接口、输出字段和确定性设置	OpenAI Codex
3	图表制作	辅助调整 MATLAB 图件的坐标、图例、字体、裁切、留白与三格式导出；图件数据只读取正式结果文件	OpenAI Codex
4	文字与排版	辅助检查术语、单位、图表引用、LaTeX 编译和官方模板一致性，并提供局部润色建议	OpenAI Codex
5	结果复核	辅助执行校验值核对、硬约束检查、主要结果定位和发布前静态审计	OpenAI Codex

F.3 关键交互记录

以下仅摘录对正式产物有代表性的交互；完整工作记录保存在项目内部台账中，且不包含账号、密钥、个人信息或未公开数据。

- 交互记录 1.** 提示词：“梳理现有 C 题产物，判断模型是否仍有优化空间，并给出可执行的收敛与验证计划。”
AI 回复摘要：按 Q1–Q4 列出候选方案、同输出基线、接受条件、风险检验和停止条件。
处理结果：参赛队逐项复核运行证据：Q1–Q3 未达到预设门槛的候选未替换主线，Q4 仅在通过硬约束和确定性复跑后保留 72 h、12 任务邻域的局部联合结果。
- 交互记录 2.** 提示词：“基于赛题结果文件用 MATLAB 重绘论文图件，修正坐标、字体、图例、裁切和留白，并导出 PDF、SVG、PNG。”
AI 回复摘要：定位结果 CSV/JSON 与 MATLAB 生成脚本，提出图型、版式和导出修改建议。
处理结果：参赛队核对源文件校验值、坐标单位、统计口径和图意后运行 MATLAB；不采用模拟数据或 AI 生成数值。
- 交互记录 3.** 提示词：“核验 Q3 完整审计 269/270 的原因，并修复为全部通过。”
AI 回复摘要：协助定位旧全时域探针的边界不一致，并建议使用含双互斥约束的区域二进制 MILP 探针重新审计。
处理结果：参赛队复核约束含义并运行正式程序，270 项审计全部通过；该结果只支持可行性与审计完整性，不新增性能或全局最优结论。
- 交互记录 4.** 提示词：“核对华数杯官方 Word 模板和 AI 使用章程，调整 LaTeX 首页、声明位置与附录结构。”
AI 回复摘要：提取首页表格、字体、字号、页边距和 AI 披露条款，形成 LaTeX 修改建议。
处理结果：参赛队按本届官方文件逐项核对，并通过 XeLaTeX 编译和 PDF 页面渲染检查后采用。

F.4 采纳和人工修改情况

表 21 AI 输出的采纳范围与人工修改

序号	AI 输出内容概述	是否采纳	人工修改情况
1	优化候选、接受条件和停止条件	部分采纳	参赛队根据题意、可计算预算和正式实验结果决定是否采用；未达条件的方案保留为未采用方案
2	程序修改与错误定位建议	部分采纳	逐段检查代码，并使用同输入基线、硬约束审计、固定种子复跑和 SHA-256 核验后才进入正式证据
3	MATLAB 图件脚本与版式建议	采纳	参赛队核对数据来源、统计定义、坐标单位、图例和结论边界，并对文字遮挡与裁切进行人工检查
4	论文局部表述与 LaTeX 排版建议	部分采纳	参赛队重新组织论证、核对公式和正式数值，并用自身语言完成最终表述
5	未经证据验证的数值、来源或结论	不采纳	不进入正文、图表、参考文献或附录；正式数字只来自赛题结果及其确定性派生量

AI 不作为作者，也不作为数学模型、公式或结论的学术论据来源；但按照竞赛章程，所用 AI 工具作为工具披露条目列入参考文献。模型选择、公式推导、证据核验、文字定稿和提交责任均由参赛队承担。